High-dimensional Portfolio Optimisation: An Empirical Study on the Maximum–Sharpe Estimated Regression

Pierre-François Pinelli Tristan Gonçalves

Supervised by **Professor Abraham Lioui**

May 2025

Abstract

Optimisation of high-dimensional mean-variance portfolio is notoriously prone to estimation errors, which can hurt out-of-sample performance. This paper proposes a robust framework that tackles the problem on two fronts: (i) a Ledoit–Wolf linear-shrinkage estimator that yields a well-conditioned covariance matrix even when the number of assets is large relative to the sample size, and (ii) the Maximum–Sharpe Estimated Regression (MAXSER) formulation, which recasts the optimisation problem as a sparse regression task and selects the ℓ_1 penalty via a K-Fold cross-validation rule tied directly to the investor's volatility target. The resulting weights are far less extreme than those of a classical Markowitz portfolio, producing tighter risk control, substantially lower transaction costs, and higher out-of-sample returns. The framework therefore marries modern high-dimensional estimation techniques with implementable portfolio construction.

Keywords: High-dimensional portfolio optimisation; Ledoit–Wolf shrinkage; MAXSER; sparse regression; covariance shrinkage; LARS solution path; transaction costs; rolling window back test.

Acknowledgements

We would like to express our deepest gratitude to **Professor Abraham Lioui**, our supervisor, for his continuous support, guidance, and encouragement throughout the duration of this project. His expertise, constructive feedback, and availability were invaluable and greatly enriched the quality of our work. We are sincerely thankful for the opportunity to learn under his supervision and for the trust he placed in us during the development of this study.

We also wish to thank **Professor Vincent Milhau** for his inspiring lectures on portfolio construction theory.

Finally, we would also like to thank everyone who contributed directly or indirectly to this project.

Contents

List of Tables

List of Figures

1 Introduction

In the past several years, high-dimensional portfolio optimisation has garnered growing interest since financial markets have become more complex and data-rich. The traditional mean-variance portfolio building approach, established by Markowitz, often exhibits suboptimal performance in high-dimensional settings because of significant estimation bias. Against this backdrop, the primary objective of this research is to assess strategies that can handle these difficulties and achieve more reliable out-of-sample risk management.

Research Questions

- How do high-dimensional settings impact the accuracy and stability of mean-variance portfolios?
- Can a combination of the MAXSER approach with the linear shrinkage method by Ledoit–Wolf significantly improve out-of-sample risk control compared to Markowitz solutions?
- To what extent does enforcing sparsity in portfolio optimisation lower cumulative transaction costs while preserving (or enhancing) risk-adjusted performance?

Methodology

We estimate portfolios in three integrated steps. First, we stabilise the sample covariance matrix with the Ledoit and Wolf linear shrinkage estimator, which keeps matrix inversions numerically reliable in high dimensions. Next, we recast the Markowitz problem as an unconstrained least squares regression, solve the full LARS Lasso path, and select the single sparsity level that meets a preset volatility target through 10-fold cross validation. Finally, we roll the procedure forward one month at a time, with a rolling window between six and twelve years, and evaluate each new weight vector over the following twelve months after deducting a proportional trading cost of ten basis points, benchmarking all net results against Markowitz strategy that relies on the raw sample covariance.

Contributions and Results

Our empirical study shows that combining Ledoit and Wolf covariance shrinkage with the sparse MAXSER optimisation delivers portfolios that dominate the sample based Markowitz benchmark on every key metric. Across rolling windows of six to twelve years the proposed approach holds volatility close to its target, raises the out of sample Sharpe ratio, and cuts transaction costs. This performance edge remains intact through the 2008 global financial crisis and the 2020 COVID-19 shock.

2 Literature Review and Theory

Before detailing the empirical design, we ground the study in the existing literature on portfolio optimisation.

2.1 Markowitz Theory

The classical mean variance framework of Markowitz describes the *efficient frontier*, namely the set of fully invested portfolios that (i) maximise expected return for a given level of risk or (ii) minimise risk for a given expected return. A point on that frontier can be computed through either one of the two equivalent optimisation problem below.

1. **Target return, variance minimisation**[1]

   Given a target return μ_p, solve:

   $$\min_{\mathbf{w}\in\mathbb{R}^N} \ \mathbf{w}^\top \Sigma \mathbf{w} \quad \text{s.t.} \begin{cases} \mathbf{1}^\top \mathbf{w} = 1, \\ \boldsymbol{\mu}^\top \mathbf{w} = \mu_p. \end{cases}$$

   (derivation in Appendix A).

2. **Target risk, return maximisation**

   Given a target risk σ_p^2, solve:

   $$\max_{\mathbf{w}\in\mathbb{R}^N} \ \boldsymbol{\mu}^\top \mathbf{w} \quad \text{s.t.} \begin{cases} \mathbf{1}^\top \mathbf{w} = 1, \\ \mathbf{w}^\top \Sigma \mathbf{w} = \sigma_p^2. \end{cases} \tag{1}$$

Including a risk-free asset. With a risk-free rate r_f the efficient frontier collapses to the Capital Market Line, a straight line through the point $(0, r_f)$.

Efficient portfolio with a risk-free asset:
Given the covariance matrix Σ of excess returns $\boldsymbol{r}$ and their mean vector $\boldsymbol{\mu}$, fix a target volatility $\sigma > 0$ and set

$$\theta = \boldsymbol{\mu}^\top \Sigma^{-1} \boldsymbol{\mu}, \qquad r^* = \sigma\sqrt{\theta}.$$

Solving $\min_{\mathbf{w}} \mathbf{w}^\top \Sigma \mathbf{w}$ subject to $\boldsymbol{\mu}^\top \mathbf{w} \geq r^*$ yields

$$\boxed{\mathbf{w}^* = \frac{\sigma}{\sqrt{\theta}} \Sigma^{-1} \boldsymbol{\mu}}$$

(derivation in Appendix C).

[1] This specific optimisation problem will not be used in our research because we are targeting σ.

2.2 MAXSER: Maximum-Sharpe Estimated Regression

The **MAXSER** framework of Ao, Li, Zheng (2019) provides an *unconstrained regression representation* equivalent to the mean-variance optimisation. N denotes the number of assets, T the sample length and $\gamma = N/T$.

Unconstrained regression representation:

Let $\mathbf{r}_t \in \mathbb{R}^N$ be excess-return vectors with population mean $\boldsymbol{\mu}$ and covariance matrix Σ. Fix $\sigma > 0$ and define

$$\theta = \boldsymbol{\mu}^\top \Sigma^{-1} \boldsymbol{\mu}, \qquad r^* = \sigma\sqrt{\theta}.$$

The constrained Markowitz problem $\min_{\mathbf{w}} \mathbf{w}^\top \Sigma \mathbf{w}$ s.t. $\boldsymbol{\mu}^\top \mathbf{w} \geq r^*$ is *exactly equivalent* to

$$\min_{\mathbf{w}} \mathbb{E}\big[(r_c - \mathbf{w}^\top \mathbf{r}_t)^2\big], \qquad r_c = \frac{1+\theta}{\sqrt{\theta}}\,\sigma, \tag{MAX-1}$$

which contains no constraints (σ appears only in the response r_c).

Bias-corrected estimation of θ:

The plug-in estimator $\hat{\theta}_s = \hat{\boldsymbol{\mu}}^\top \hat{\Sigma}^{-1} \hat{\boldsymbol{\mu}}$ suffers a strong upward bias as soon as the dimension ratio $\gamma = N/T$ tends towards one. Kan and Zhou (2007) propose the following *unbiased* alternative, valid whenever $0 < \gamma < 1$, built on the incomplete beta function:

$$\hat{\theta}_{\mathrm{KZ}} = \frac{(T-N-2)\,\hat{\theta}_s - N}{T} + \frac{2\,\hat{\theta}_s^{N/2}\left(1+\hat{\theta}_s\right)^{-(T-2)/2}}{T\,B_{\hat{\theta}_s/(1+\hat{\theta}_s)}\left(\frac{N}{2}, \frac{T-N}{2}\right)}, \tag{KZ}$$

where $B_x(a,b) = \int_0^x t^{a-1}(1-t)^{b-1}\,dt$ is the *incomplete beta function*. The extra term guarantees positivity and greatly reduces finite-sample variance.
Hence, yielding the response $\widehat{r}_c$:

$$\widehat{r}_c \;=\; \frac{1+\hat{\theta}_{\mathrm{KZ}}}{\sqrt{\hat{\theta}_{\mathrm{KZ}}}}\,\sigma.$$

When $\gamma \geq 1$ the unbiasedness property of $\hat{\theta}_{\mathrm{KZ}}$ no longer holds. To solve this issue, a new high-dimension correction for θ will therefore be developed in Section 2.4.

Sparse regression with an ℓ_1 penalty:

Using $\widehat{r}_c$ and the returns we solve

$$\widehat{\mathbf{w}} = \arg\min_{\mathbf{w}} \frac{1}{T}\sum_{t=1}^{T}\big(\widehat{r}_c - \mathbf{w}^\top \mathbf{r}_t\big)^2 \quad \text{s.t. } \|\mathbf{w}\|_1 \leq \lambda, \tag{MAX-2}$$

with λ chosen by 10-fold cross-validation so that the realised volatility of $\widehat{\mathbf{w}}$ matches the target σ.

Important Consideration. Steps (MAX-1) and (MAX-2) show that MAXSER relies on two building blocks: (i) an accurate inverse covariance matrix and (ii) a stable bias-corrected Sharpe-squared estimator $\hat{\theta}$. Both become fragile when the ratio $\gamma = N/T > 1$, precisely the regime faced in modern equity universes.
The following section details how high-dimensionality undermines the plug-in Markowitz solution and motivates the shrinkage and regularisation techniques we adopt thereafter.

2.3 High-dimensional Portfolio Challenges

Let N be the number of assets and T the sample length. When the dimension ratio $\gamma = N/T$ is not negligibly small, the plug-in Markowitz solution faces two major difficulties.

1. Noisy and nearly singular covariance estimates

The sample covariance matrix $\hat{\Sigma}$ contains $N(N+1)/2$ parameters so its estimation error grows quadratically in N while T grows only linearly. Its smallest eigenvalues are biased toward zero and $\hat{\Sigma}$ becomes singular as soon as $N \geq T$.

2. Explosive weights and over-fitting

Small eigenvalues of $\hat{\Sigma}$ are strongly amplified in the weights $\hat{\mathbf{w}} = \dfrac{\sigma}{\sqrt{\hat{\theta}}}\,\hat{\Sigma}^{-1}\hat{\boldsymbol{\mu}}$, producing extreme positions that track noise rather than genuine premia out of sample.

Implication. High-dimensionality causes severe risk of bias in the estimation. We address this problem using the covariance linear shrinkage method of Ledoit and Wolf (2004), introduced in the next section.

2.4 Ledoit and Wolf Linear Shrinkage Estimator

Setting:

Let $\{\mathbf{x}_t\}_{t=1}^{T} \subset \mathbb{R}^N$ be centred excess returns and $\hat{\Sigma}$ the sample covariance matrix:

$$\hat{\Sigma} \;=\; \frac{1}{T}\sum_{t=1}^{T}\mathbf{x}_t\mathbf{x}_t^{\top}$$

Linear shrinkage idea:
Ledoit and Wolf propose to combine $\hat{\Sigma}$ with a structured target F by the convex blend

$$\hat{\Sigma}_{\mathrm{LW}} \;=\; \delta\, F + (1-\delta)\,\hat{\Sigma}, \qquad 0 \leq \delta \leq 1.$$

The optimal weight δ^* minimises the expected Frobenius loss $\mathbb{E}\|\hat{\Sigma}_{\mathrm{LW}} - \Sigma\|_F^2$ and admits a closed-form expression that can be estimated consistently from the data.

Choice of the target F:
Scikit-learn implements `LedoitWolf` with the scalar target $F = \mu\, I_N$ where $\mu = \frac{1}{N}\,\mathrm{tr}(\hat{\Sigma})$. This corresponds to shrinking the sample matrix toward an identity whose diagonal matches the average variance. We adopt this for three practical reasons:

1. It requires no extra tuning and is available in a single function call, hence it integrates seamlessly into our implementation.

2. Ledoit and Wolf show that the identity target already yields a substantial reduction in estimation error in large dimensions, leading to better conditioned inverses and more stable portfolios.

3. In preliminary tests it delivered lower realised volatility and turnover than the plain sample covariance, consistent with the original findings.

Alternative targets:
One may shrink toward the diagonal of the sample covariance, a multi-factor structure, or other economically motivated forms. Such variants can further reduce noise but they require additional research and lie outside the scope of the present study.

Resulting estimator:
With the identity target the data-driven weight $\hat{\delta}$ computed by scikit-learn produces

$$\boxed{\hat{\Sigma}_{\mathrm{LW}} \;=\; \hat{\delta}\,\mu\, I_N + (1-\hat{\delta})\,\hat{\Sigma}}$$

which is positive definite for every finite N and T and can therefore be inverted reliably inside the MAXSER routine.

Summary:
We use `sklearn.covariance.LedoitWolf` with its default parameters. This choice shrinks the sample covariance toward a scaled identity matrix, balances bias and variance automatically, and preserves computational simplicity. Exploring alternative targets such as $\mathrm{diag}(\hat{\Sigma})$ is left for future work.

The estimator completes the theoretical part. The next section shows how we integrate it into a practical workflow.

3 Methodology

3.1 Our Approach

Our methodology combines three complementary building blocks that, taken together, yield robust high-dimensional portfolios:

1. **Ledoit–Wolf shrinkage for covariance stabilisation:**

   The "naive" sample covariance matrix is replaced by the linear-shrinkage estimator $\widehat{\Sigma}_{\text{LW}}$ (Section 2.4). All subsequent statistics, most importantly the inverse covariance that enters the tangency Sharpe ratio θ, are therefore computed from $\widehat{\Sigma}_{\text{LW}}$ for our implementation.

2. **MAXSER reformulation with sparse regression:**

   The risk-budget Markowitz problem is recast as an unconstrained regression followed by an ℓ_1-penalised fit (equations MAX-1 and MAX-2). Solving the full *LARS* path delivers the entire family $\{\mathbf{w}(\lambda)\}$ in at most N iterations, producing parsimonious and interpretable portfolios even when $N \geq T$.

3. **K-Fold cross-validation for selecting λ:**

   We split the T observations into K (nearly) equal folds.

   Then for each fold $i = 1, \ldots, K$:

   (a) *Training set:* all data *except* the ith fold. Fit the LARS–Lasso path on this training set, which returns a sequence of m knots $\left\{\left(\alpha^{(j)},\, w^{(j)}\right)\right\}_{j=1}^{m}$, where $\alpha^{(j)}$ is the penalty at step j and $w^{(j)} \in \mathbb{R}^N$ the corresponding weight vector.

   (b) *Validation covariance:* compute Ledoit–Wolf estimate $\Sigma_{\text{val}}^{(i)}$ on the held-out fold.

   (c) *Realised volatility for each knot:*

   $$\text{vol}_j = \sqrt{\left(w^{(j)}\right)^{\top} \Sigma_{\text{val}}^{(i)}\, w^{(j)}}, \quad j = 1, \ldots, m.$$

   (d) *Select best knot:* choose

   $$j_i^* = \arg\min_{1 \leq j \leq m} \left|\text{vol}_j - \sigma_{\text{target}}\right|, \qquad \lambda_i = \alpha^{(j_i^*)}.$$

   After cycling through all K folds, aggregate the fold-wise winners by

   $$\widehat{\lambda} \;=\; \frac{1}{K} \sum_{i=1}^{K} \lambda_i.$$

This K-fold cross-validation procedure splits the estimation sample into K disjoint, (nearly) equal-sized folds, using each fold once as the validation set and the remaining $K-1$ folds for training. By measuring the realised volatility $\sqrt{w^\top \Sigma_{\text{val}}\, w}$ on each held-out fold, we ensure that the selected penalty $\widehat{\lambda}$ is driven by true out-of-sample performance. In particular: (i) Every observation serves exactly once for validation, which prevents tuning to noise in any one region. (ii) The volatility on each fold is computed on data unseen during its fit, so $\widehat{\lambda}$ directly targets the ex-ante risk level. (iii) Averaging the K fold-wise penalties yields a stable $\widehat{\lambda}$ that balances bias and variance, guarding against overfitting the training set's volatility.

Benchmark comparison: naive Markowitz portfolio.

MAXSER is benchmarked against the simplest feasible alternative that shares the same risk budget but intentionally avoid shrinkage and sparsity: the *naive Markowitz portfolio* obtained from the *sample* covariance as,

$$\boxed{\mathbf{w}_{\text{naive}} \;=\; \frac{\sigma_{\text{target}}}{\sqrt{\hat{\theta}_{\text{naive}}}}\,\widehat{\Sigma}^{-1}\hat{\mu}} \qquad \hat{\theta}_{\text{naive}} = \hat{\mu}^\top \widehat{\Sigma}^{-1}\hat{\mu},$$

where $\widehat{\Sigma}$ and $\hat{\mu}$ are the sample estimators computed on the current estimation window.

The full pipeline Ledoit–Wolf estimation, bias-corrected Sharpe calculation, K-Fold choice of $\widehat{\lambda}$, sparse weight estimation, and rolling out-of-sample evaluation appears in `solutionpath.py` and `rollingwindow.py` (described in details later in 3.3). Only the constant risk-free rate and the target volatility σ_{target} are fixed exogenously; every other tuning parameter is determined automatically within each estimation window.

Practical note on matrix inversion:

In high-dimensional settings the sample covariance $\widehat{\Sigma}$ is often singular ($N \geq T$) or ill-conditioned (very small eigenvalues). To guarantee that *every* linear system involving $\widehat{\Sigma}$ can be solved, we systematically replace the ordinary inverse $\widehat{\Sigma}^{-1}$ by its *(Moore–Penrose) pseudo-inverse*, denoted $\widehat{\Sigma}^{\dagger}$. Formally, for an arbitrary real matrix $A \in \mathbb{R}^{m\times n}$ with singular-value decomposition $A = U\,\mathrm{diag}(\sigma_1,\ldots,\sigma_r)\,V^\top$ ($r = \operatorname{rank} A$), the pseudo-inverse

$$A^{\dagger} \;=\; V\;\mathrm{diag}\Big(\tfrac{1}{\sigma_1},\ldots,\tfrac{1}{\sigma_r}\Big)\,U^\top$$

is the **unique** matrix that satisfies the four Moore–Penrose conditions: $AA^{\dagger}A = A$, $A^{\dagger}AA^{\dagger} = A^{\dagger}$, $(AA^{\dagger})^\top = AA^{\dagger}$, $(A^{\dagger}A)^\top = A^{\dagger}A$. When A is square and full rank, $A^{\dagger}$ coincides with A^{-1}; otherwise it yields the *minimum-norm least-squares solution* to $Ax = b$ in closed form, $x = A^{\dagger}b$. In the section 3.3, Algorithm 1 turns our approach into a routine that runs *inside every rebalancing window.*

3.2 Why did we choose this approach?

Why using shrinkage inside MAXSER?

The MAXSER algorithm relies on Σ^{-1} twice: first to form the squared tangency Sharpe ratio $\theta = \boldsymbol{\mu}^\top \Sigma^{-1} \boldsymbol{\mu}$ and, second, inside the sparse regression that assigns portfolio weights. Unfortunately the sample matrix $\hat{\Sigma}$ is often ill conditioned (or even singular) when the dimension ratio $\gamma = N/T$ is large. Substituting the Ledoit–Wolf estimate $\hat{\Sigma}_{\text{LW}}$ from (2.4) resolves these difficulties in one stroke. Because $\hat{\Sigma}_{\text{LW}}$ is positive definite, its inverse exists for every sample and possesses a moderate condition number (small input errors only cause small solution errors), yielding numerically stable matrix operations. The same substitution also moderates the upward bias of the plug-in Sharpe-squared statistic $\hat{\theta}_{LW} = \hat{\boldsymbol{\mu}}^\top \hat{\Sigma}_{\text{LW}}^{-1} \hat{\boldsymbol{\mu}}$, and empirical studies show that weights computed with $\hat{\Sigma}_{\text{LW}}$ exhibit lower realised variance and turnover than their sample-covariance counterparts. For these reasons the Ledoit–Wolf shrinkage estimator is employed throughout every step of our procedure.

Why the MAXSER problem is solved with LARS?

The MAXSER problem converts the risk-budget Markowitz problem into an *unconstrained regression representation* equivalent to the mean-variance optimisation. Among available solvers, *Least-Angle Regression* (LARS) is chosen by Ao, Li and Zheng (2019) because it produces the entire solution path $\{\mathbf{w}(\lambda) : \lambda \geq 0\}$ in at most N iterations, so the portfolio that matches the target volatility σ can be selected without re-running the optimisation for multiple tuning parameters. It returns sparse, interpretable portfolios, after k steps at most k assets carry non-zero positions, and it avoids internal cross-validation, reducing computational cost.

Why centering collapses the LARS path?

The LARS algorithm initiates with the zero portfolio $\mathbf{w} = \mathbf{0}$ and defines the initial residual as

$$e_t^{(0)} \;=\; r_c - \mathbf{0}^\top \mathbf{r}_t = r_c, \qquad t = 1, \ldots, T,$$

which is constant across all observations. In the first step, LARS computes the empirical correlation between each predictor r_{tj} and the residual:

$$\widehat{\text{Cor}}\big(r_{tj},\, e_t^{(0)}\big) \;=\; \frac{1}{T} \sum_{t=1}^{T} r_{tj}\, r_c \;=\; r_c\, \bar{r}_j,$$

where $\bar{r}_j = \frac{1}{T} \sum_t r_{tj}$ is the sample mean of predictor j. Since $\bar{r}_j$ is generally nonzero, at least one predictor will exhibit a nonzero correlation with the residual, allowing LARS to initiate a sparse solution path by moving along that direction.

If the predictors are first centred, such that

$$\tilde{r}_{tj} = r_{tj} - \bar{r}_j, \qquad j = 1, \ldots, N,$$

then

$$\frac{1}{T}\sum_{t=1}^{T} \tilde{r}_{tj}\, r_c \;=\; r_c \cdot \frac{1}{T}\sum_{t=1}^{T} \tilde{r}_{tj} \;=\; r_c \cdot 0 = 0 \qquad \text{for every } j.$$

Consequently, all initial correlations are zero, and the LARS algorithm terminates immediately with the trivial solution $\mathbf{w} = \mathbf{0}$ for all values of the regularisation parameter λ. This eliminates the sequence of sparsity patterns that normally provides interpretability along the solution path.

Avoiding centering preserves the structure of the residuals, enables LARS to initiate its stepwise construction of the coefficient path, and does not affect the final estimates, since the model contains no intercept.

3.3 Algorithms

Lines **1–4:** Estimate full-sample statistics. Compute the Ledoit–Wolf covariance Σ_{full}, the sample mean μ, and the tangency Sharpe $\hat{\theta}_{\text{LW}} = \mu^\top \Sigma_{\text{full}}^{-1} \mu$.

Lines **5–7:** Set the target return $\hat{r}_c = \sigma_{\text{target}} (1 + \hat{\theta}_{\text{LW}}) / \sqrt{\hat{\theta}_{\text{LW}}}$, $\quad y_{\text{full}} = \hat{r}_c \mathbf{1}_T$.

Lines **8–17:** Perform standard K-fold CV. For each fold:

- Fit the LARS–Lasso path on the training folds, yielding $\{(\alpha^{(j)}, w^{(j)})\}_{j=1}^{m}$.
- Estimate Σ_{val} on the held-out fold.
- Compute realized volatility $\sqrt{(w^{(j)})^\top \Sigma_{\text{val}} w^{(j)}}$ for each knot.
- Select the penalty $\lambda_i = \alpha^{(j^*)}$ minimizing $|\text{vol} - \sigma_{\text{target}}|$ and record it.

Line **18:** Aggregate the K fold-wise penalties by $\hat{\lambda} = \frac{1}{K} \sum_{i=1}^{K} \lambda_i$.

Lines **19–24:** Refit LARS–Lasso on the full sample, then pick the knot whose λ is closest to $\hat{\lambda}$. The corresponding w is w_{final}.

Algorithm 1 MAXSER Portfolio via LARS–Lasso and K–Fold Cross-Validation

Require: return matrix $R \in \mathbb{R}^{T \times N}$, target volatility σ_{tgt}, number of folds K
Ensure: weight vector $w^* \in \mathbb{R}^N$ that attains σ_{tgt} on average

function MAXSER($R, \sigma_{\text{tgt}}, K$)
  $\Sigma \leftarrow$ LedoitWolf(R) ▷ linear shrinkage covariance
  $\mu \leftarrow$ columnMean(R)
  $\hat{\theta}_{\text{LW}} \leftarrow \mu^\top \Sigma^{-1} \mu$
  $\hat{r}_c \leftarrow \sigma_{\text{tgt}} (1 + \hat{\theta}_{\text{LW}}) / \sqrt{\hat{\theta}_{\text{LW}}}$
  $y \leftarrow (\hat{r}_c, \ldots, \hat{r}_c)^\top \in \mathbb{R}^T$ ▷ constant response
  $\Lambda \leftarrow [\,]$ ▷ store best λ from each fold
  for each split $(\mathcal{I}_{\text{tr}}, \mathcal{I}_{\text{val}})$ produced by KFold(T, K) **do**
    $X_{\text{tr}} \leftarrow R[\mathcal{I}_{\text{tr}}]$
    $X_{\text{val}} \leftarrow R[\mathcal{I}_{\text{val}}]$
    $y_{\text{tr}} \leftarrow (\hat{r}_c, \ldots, \hat{r}_c)^\top$ ▷ same length as X_{tr}
    $(\alpha, \text{w}) \leftarrow$ LARSPath($X_{\text{tr}}, y_{\text{tr}}$) ▷ α: vector of regularisation strength, w_j: weight vector at knot j
    $\Sigma_{\text{val}} \leftarrow$ LedoitWolf(X_{val})
    for $j = 1$ **to** length(α) **do**
      $v_j \leftarrow \sqrt{w_j^\top \Sigma_{\text{val}} w_j}$ ▷ validation volatility
    end for
    $j^\star \leftarrow \arg\min_j \left| v_j - \sigma_{\text{tgt}} \right|$
    $\Lambda \leftarrow \Lambda \cup \{\alpha_{j^\star}\}$
  end for
  $\hat{\lambda} \leftarrow \frac{1}{K} \sum_{i=1}^{K} \Lambda_i$ ▷ aggregate regularisation strength
  $(\alpha^{\text{full}}, w^{\text{full}}) \leftarrow$ LARSPath(R, y)
  $j^\dagger \leftarrow \arg\min_j \left| \alpha_j^{\text{full}} - \hat{\lambda} \right|$
  $w^* \leftarrow w_{j^\dagger}^{\text{full}}$
  return w^*
end function

Table 1 gives the precise index arithmetic for the K-fold CV splits used throughout. Each of the K folds has a size $L = \lfloor T/K \rfloor$, so that every observation appears exactly once in a validation fold. For fold $i = 1, \ldots, K$, the validation indices are $[(i-1)\,L + \min(i-1, R)\,,\ i\,L + \min(i, R)\ -\ 1]$, and the training set is its complement. This scheme ensures, (i) balanced use of data for both training and validation, (ii) no overlap between hold-out sets, and (iii) clear, reproducible splits with minimal remainder handling.
The combination of Algorithm 1 and Table 1 therefore constitutes a fully specified recipe for producing the weight sequence $\{w_{\text{final}}(t)\}_{t=1}^{T_{\text{test}}}$ against which all results in Section 4 are benchmarked.

Table 1: K-Fold CV Partitioning

Concept	**Details**
Partition into segments	Let T be the total number of observations. For K folds, compute $$L = \left\lfloor \frac{T}{K} \right\rfloor = \left\lfloor \frac{100}{10} \right\rfloor = 10, \quad R = T \bmod K = 0.$$ This yields K folds of size $L = 10$.
Fold definition	For each fold $i = 1, \ldots, K$, set $$s_i = L = 10, \quad \text{val. indices} = \big[(i-1)\,L,\ i\,L - 1\big],$$ $$\text{train indices} = \{0, 1, \ldots, 99\} \setminus \text{val. indices}\,.$$
Shuffle (optional)	If `shuffle=True`, randomly permute the data before computing the above segments.
Leftover observations	None: by construction every observation appears exactly once in a validation set.
Illustration ($T = 100$, $K = 10$)	All folds have size 10: • Fold 1: train 10–99, val 0–9 • Fold 2: train 0–9 ∪ 20–99, val 10–19 • ... • Fold 10: train 0–89, val 90–99

With the methodology fully explained, we now turn to its empirical performance.

4 Results

4.1 Data

Our empirical analysis relies on two primary datasets: (i) a broad cross-section of Fama–French portfolio returns and (ii) individual U.S. stock returns from Wharton Research Data Services (WRDS) CRSP. We describe the data sources, sample periods, and processing steps below, together with their roles in the rolling-window experiments.

Fama–French Portfolios (1980–2024):

Using `pandas_datareader`'s `famafrench` service we download five benchmark return panels from the Kenneth R. French data library for the full span *January 1980 – December 2024*:

1. `Portfolios_Formed_on_BE-ME` (book-to-market),
2. `48_Industry_Portfolios`,
3. `Portfolios_Formed_on_INV` (total investment),
4. `100_Portfolios_10x10` (size × value),
5. `100_Portfolios_ME_OP_10x10` (size × operating profitability).

Each download returns monthly total returns in percent. All five matrices are concatenated column-wise and any month containing a missing value in *any* series is dropped, yielding a balanced panel of roughly $T = 540$ months and $N = 285$ portfolios. Returns are converted from percent to decimals and transformed to *excess* form, $r_{i,t}^{\text{ex}} = r_{i,t} - r_f$, by subtracting a constant risk-free rate of 2% per annum ($\approx 0.165\%$ per month). No additional data pre-processing is applied.

Individual Stocks (1990–2024):

To evaluate the strategies on actual equities we construct a CRSP panel of large-cap U.S. stocks via WRDS. Starting from an initial universe of 182 tickers (detailed in the code in Appendix G), we pull monthly total returns for *January 2000 – December 2024* and *January 1990 – December 2024*. A survivorship filter removes any stock with a missing observation within this period, we have $N = 182$ stocks and $T = 300$ months for the *2000-2024* period and leaving $N = 112$ stocks and $T = 420$ months for the *1990-2024* period. Returns are already in decimal form; we subtract the same constant risk-free rate (0.165% per month) to obtain excess returns.

Rolling Window Usage and Data Splitting:

The two datasets serve complementary purposes:

- **`solutionpath.py`.** The high-dimensional Fama–French panel is used mainly for simple *in-sample* and *out-of-sample* illustrations (e.g. tracing the full LARS solution path, and for train/test splits that shows solution path out-of-sample).

- **`rollingwindow.py`.** The CRSP stock panel underpins a full rolling window back-test. At each month t we use the most recent W_{est} months as the estimation window, re-estimate the portfolio (naive minimum-variance or MAXSER), and apply the weights to month $t+1$. The procedure advances one month at a time from *1990/01* or *2000/01* to *2024/12*, generating a number of out-of-sample observations depending on the size on the window W_{est}. Realised performance metrics are computed over the next 12 month as $W_{\text{eval}} = 12$-month evaluation blocks. A target volatility of $\sigma = \sigma_{target}$ (8% monthly in our implementation) is imposed for both strategies.

Summary:

The Fama–French portfolios provide a long, high-dimensional history for method calibration and visualisation, whereas the CRSP stock data supply a realistic out-of-sample test. Both datasets are expressed in excess-return form using a constant risk-free rate, ensuring comparability across all empirical exercises. The *1990–2024* period includes fewer stocks ($N = 112$) and is primarily used to evaluate the robustness of the strategy across major market crisis, including the Global Financial Crisis of 2008 and the COVID-19 crash. The *2000–2024* period contains more stocks ($N = 182$) and is used to assess the impact of the window size and sample length on performance and stability, particularly in high-dimensional regimes where $\gamma' = N/W_{est}$ is high.

In the following part, we use the high-dimensional Fama–French panel to visualise how the estimators behave.

4.2 Solution Path Illustrations

We begin by illustrating the MAXSER solution path using the Fama–French portfolios. The goal is to track how the *risk* (standard deviation) and *Sharpe ratio* evolve across the sparse solutions generated by the `lars_path` algorithm.

Let $R \in \mathbb{R}^{T \times N}$ be the matrix of monthly excess returns and $\{\mathbf{w}_j\}_{j=0}^{m}$ the sequence of portfolios computed along the LARS path. For each $\mathbf{w}_j$, we compute

$$\zeta = \frac{\|\mathbf{w}\|_1}{\|\mathbf{w}_{\text{OLS}}\|_1},$$

where $\mathbf{w}_{\text{OLS}}$ is the ordinary least-squares solution to $\min_{\mathbf{w}} \|\mathbf{y} - \mathbf{w}^\top R\|_2^2$. This normalised ℓ_1 ratio satisfies $\zeta = 0$ for the all-zero vector and $\zeta = 1$ for the dense OLS solution. Each value of ζ thus corresponds to a level of sparsity along the MAXSER path.

We define the corresponding portfolio return vector

$$r_p = R\,\mathbf{w}_j$$

and denote its sample mean by

$$\bar{r}_p = \frac{1}{T}\sum_{t=1}^{T} r_{p,t}.$$

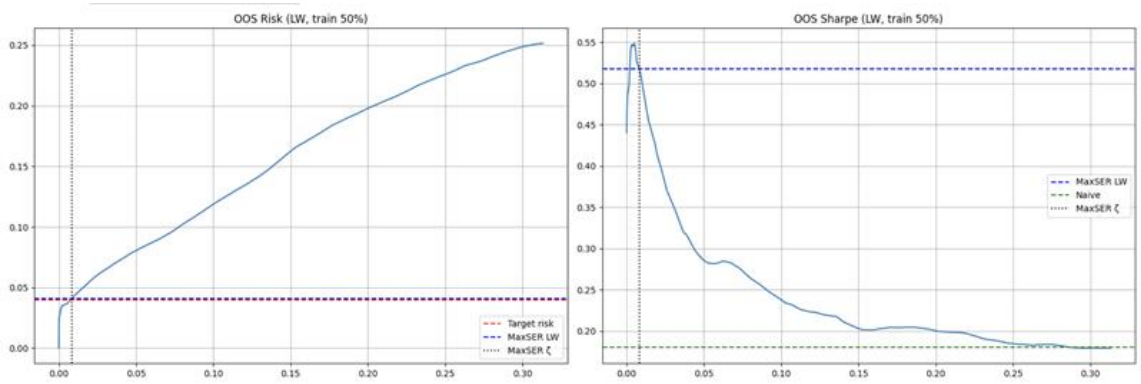
The risk and Sharpe ratio associated with sparsity level ζ are given by:

$$\text{SD}(\zeta) = \sqrt{\frac{1}{T}\sum_{t=1}^{T}\left(r_{p,t} - \bar{r}_p\right)^2}, \tag{1}$$

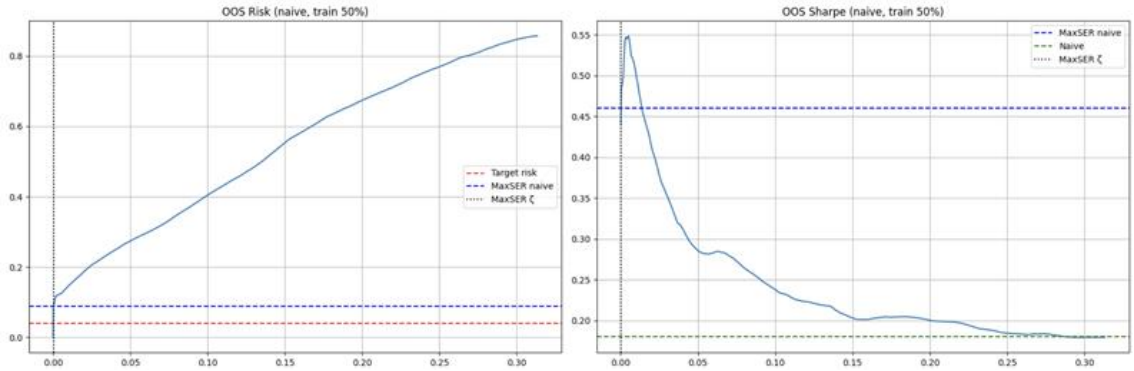
$$\text{SR}(\zeta) = \frac{\bar{r}_p}{\text{SD}(\zeta)}. \tag{2}$$

This construction enables a visual diagnosis of the reward–risk trade-off along the entire LARS path. By plotting $\text{SD}(\zeta)$ and $\text{SR}(\zeta)$ as functions of ζ, we assess how quickly the portfolio recovers target volatility and whether performance gains plateau or deteriorate as the solution becomes denser. As shown in the figures that follow, the MAXSER method achieves most of its Sharpe ratio improvement within a small fraction of the ℓ_1 budget, supporting the idea that only a limited number of active positions are needed to approach the efficient frontier.

Out-of-sample comparison between covariance estimators:
Figure 1 displays the performance of MAXSER when the covariance matrix is estimated with the Ledoit–Wolf shrinkage formula (Panel 1a) and when the raw sample covariance is used instead (Panel 1b). The training window always covers one half of the available observations while the remaining half serves as the test set. With the shrinkage matrix the LARS path meets the prescribed volatility almost exactly and maintains a Sharpe ratio that is well above the benchmark for every sparsity level. The portfolio based on the sample matrix overshoots the risk target . These results confirm that a well conditioned covariance estimate is essential for stable risk control when the dimension ratio is large. We therefore adopt the Ledoit–Wolf estimator throughout the rest of the study.[2]



(a) Ledoit–Wolf covariance, training share 50%



(b) Sample covariance, training share 50%

Figure 1: Out of sample risk (left) and Sharpe ratio (right) along the MAXSER LARS path under two covariance estimators. The vertical dotted line marks the sparsity level selected by expanding time series cross validation. The horizontal dashed lines show the volatility target and the realised performance of MAXSER at the selected knot.

[2]For brevity we include only the 65 % training–share comparison in Appendix D. The companion script `solutionpath.py` (listed in Appendix G) re-creates identical risk and Sharpe-ratio curves for every split from 10 % to 90 %; all splits confirm the superiority of the Ledoit–Wolf specification.

Commentary. Two messages emerge. First, MAXSER combined with Ledoit–Wolf keeps realised volatility on target while the naive alternative exceeds the limit by a wide margin. Second, the Sharpe ratio reaches its maximum for very sparse portfolios, and the shrinkage variant stays above the naive benchmark across the entire path.

In-sample Results:

After establishing that the Ledoit–Wolf method dominates its naïve counterpart out of sample, we turn to a diagnostic look at the behaviour of MAXSER *inside* the estimation window. The next two figures trace the entire LARS solution path and report, for each sparsity level ζ, the in-sample volatility and Sharpe ratio that would have been observed had the corresponding portfolio been implemented immediately. These curves clarify how quickly the estimator attains the prescribed risk budget and how many active positions are required before additional assets cease to improve performance.

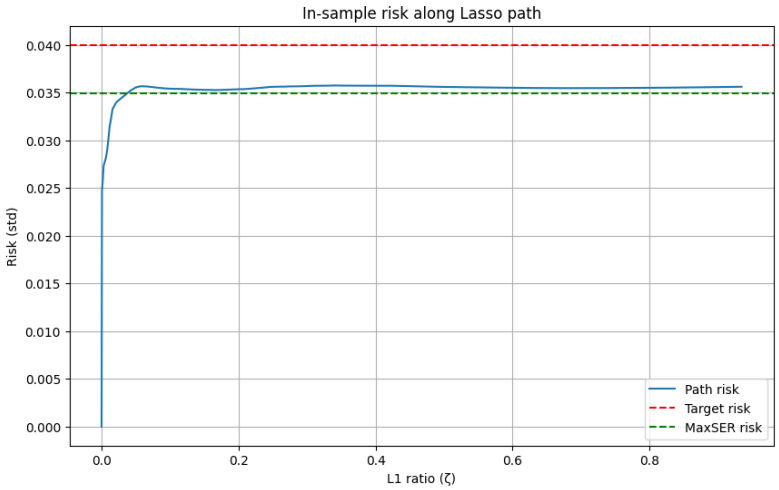


Figure 2: In-sample risk (standard deviation) along the MAXSER solution path. The horizontal red line marks the 4 % monthly volatility target.

Commentary. Figure 2 shows that the realised standard deviation rises sharply from $\zeta = 0$ (zero weights) and stabilises just below the 4% target after around 5–10% of the ℓ_1 budget has been recovered. Increasing the budget beyond that point yields little gain in variance control, once more suggesting that a small active subset captures most of the risk-reducing signal.

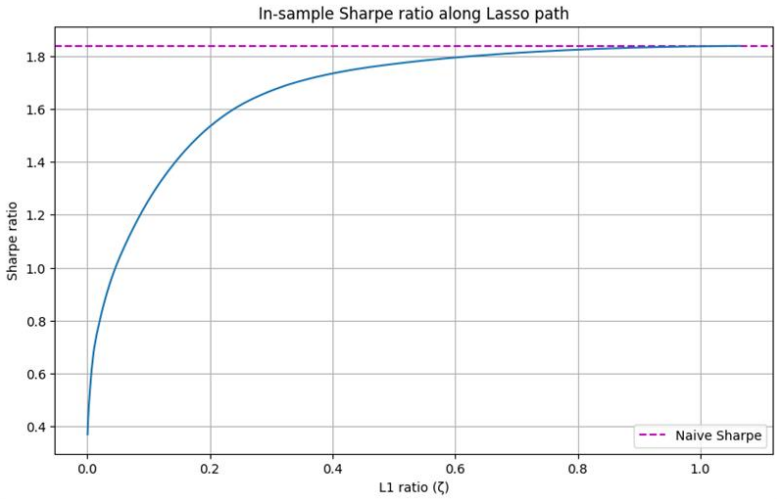


Figure 3: In-sample Sharpe ratio along the MAXSER solution path.

Commentary. Figure 3 reveals a steady improvement in Sharpe ratio as the portfolio becomes less sparse. The ratio increases sharply between $\zeta = 0$ and $\zeta \approx 0.30$, before levelling off beyond $\zeta \approx 0.50$. By that point the volatility constraint has already been met and only a limited number of assets are active. The selected MAXSER solution achieves nearly optimal risk-adjusted return while maintaining a sparse, stable allocation.

The intuition gathered from the solution path must hold out-of-sample with portfolio rebalancing. We test this in the rolling window experiment in the next part.

4.3 Rolling Window Results

We now assess the sensitivity of the MAXSER strategy to the choice of rolling window. We compare multiple estimation windows while keeping the evaluation period fixed at 12 months. The target is set at a monthly volatility of 8%. Results are split by sample period: the 2000–2024 period includes a number of stocks ($N = 182$), while the 1990–2024 period uses a smaller set ($N = 112$) to ensure availability of returns throughout.

2000–2024 period ($N = 182$). Figure 5 compares 96-month and 144-month estimation windows. In both cases, MAXSER consistently outperforms the naive benchmark in terms of risk control and Sharpe ratio. On top of that, the naive benchmark struggle to stay close to the 8% volatility target while MAXSER reaches the target risk on average.

Summary statistics. Table 2 summarises the average volatility and Sharpe ratio obtained with each setting over the 2000-2024 period.

Estimation Window (years)	Volatility		Sharpe Ratio	
	Naive	MAXSER	Naive	MAXSER
96 (8)	0.181213	0.090137	0.178249	0.289403
108 (9)	0.196947	0.081934	0.251739	0.294124
120 (10)	0.230283	0.081164	0.256071	0.282336
132 (11)	0.289564	0.080765	0.231357	0.305890
144 (12)	0.389607	0.082897	0.174722	0.241245

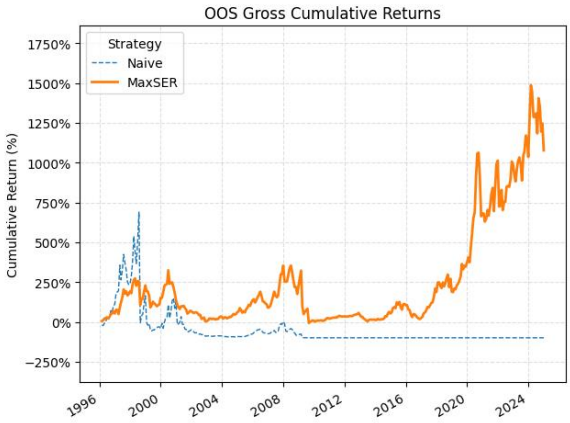
Table 2: Mean Out-of-Sample Metrics (2000-2024) by Estimation Window ($N = 182$)

1990–2024 period ($N = 112$). To evaluate robustness across different market regimes, we turn to the longer 1990–2024 sample and examine performance around the 2008 global financial crisis and the 2020 COVID-19 shock. We use 72-month and 108-month windows. These windows show that MAXSER can go through the 2008 and 2020 crisis and still meet the target volatility, while the naive skyrocketed up to 176% of volatility, showcasing the explosive weight behavior.

Summary statistics. Table 3 summarises the average volatility and Sharpe ratio obtained with each setting over the 1990-2024 period.

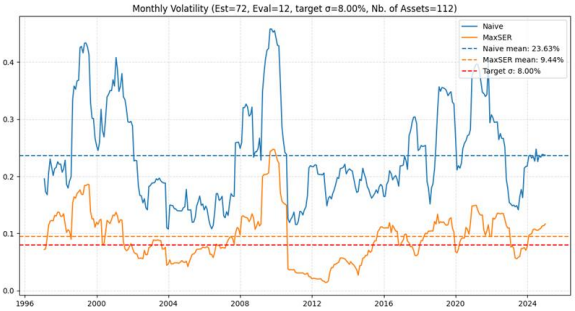
Estimation Window (years)	Volatility		Sharpe Ratio	
	Naive	MaxSER	Naive	MaxSER
72 (6)	0.236337	0.094420	0.108719	0.177816
84 (7)	0.344998	0.085899	0.093630	0.163218
96 (8)	0.608153	0.082056	0.015376	0.200892
108 (9)	1.766684	0.082490	-0.024828	0.150057

Table 3: Mean Out-of-Sample Metrics (1990–2024) by Estimation Window ($N = 112$)



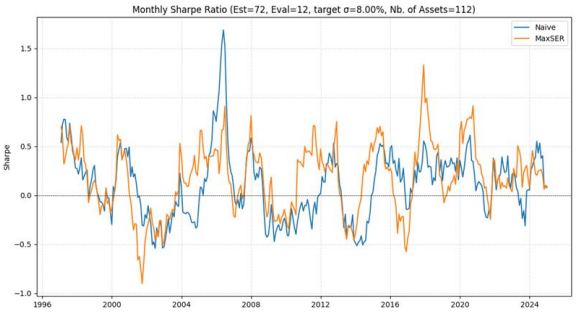
(a) Cumulative returns (72-month window)

(b) Cumulative returns (108-month window)

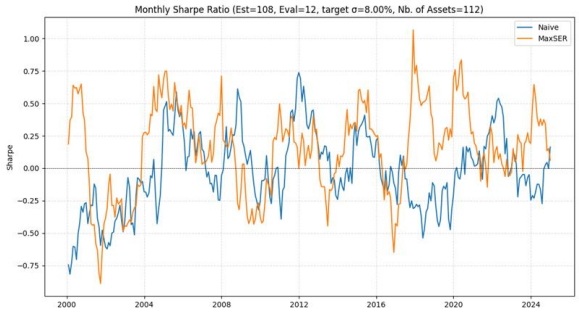


(c) Volatility (72-month window)

(d) Volatility (108-month window)

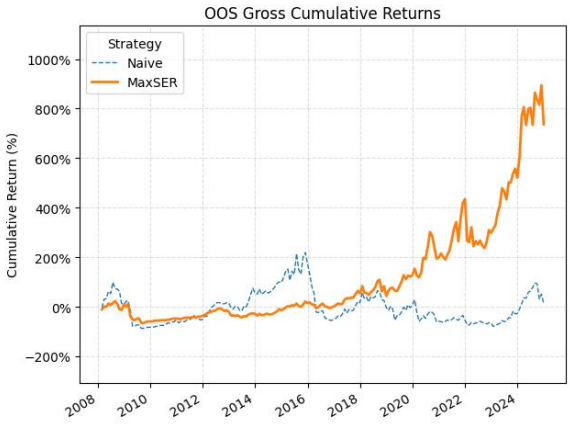


(e) Sharpe ratio (72-month window)

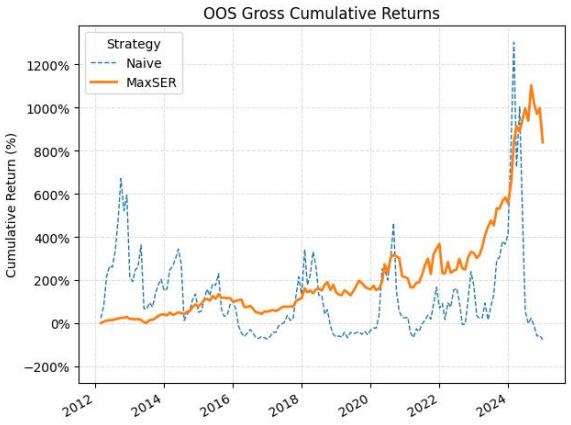


(f) Sharpe ratio (108-month window)

Figure 4: Performance of MAXSER strategy over the 1990–2024 period under different rolling windows. This setting uses a universe of $N = 112$ assets.

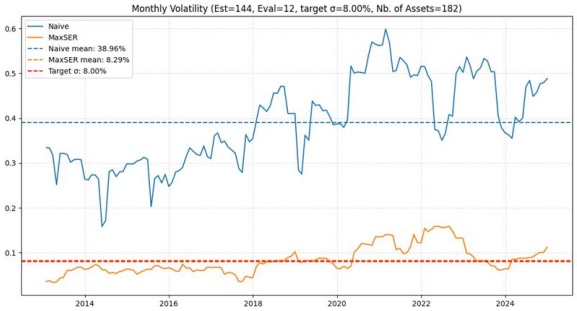


(a) Cumulative returns (96-month window)

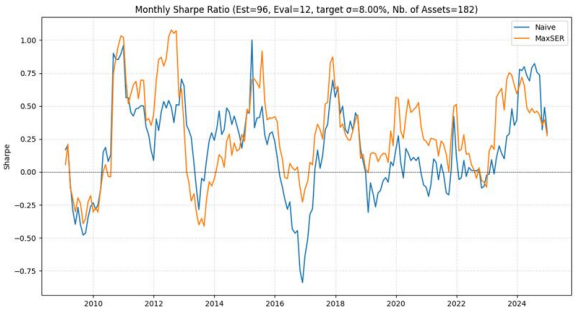


(b) Cumulative returns (144-month window)

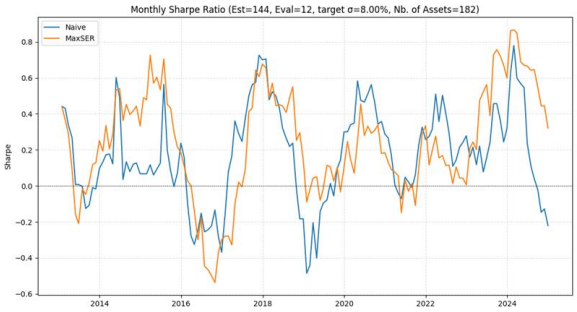
(c) Volatility (96-month window)



(d) Volatility (144-month window)



(e) Sharpe ratio (96-month window)



(f) Sharpe ratio (144-month window)

Figure 5: Performance of MAXSER strategy over the 2000–2024 period under different rolling estimation windows. The naïve benchmark is shown in dashed blue.[3]

The remaining window configurations are included in Appendix E (for 2000–2024) and Appendix F (for 1990–2024).

Finally, we quantify how these allocations translate into real-world trading costs.

[3]Be aware that final cumulative return highly depends on the time portfolio is first introduced (e.g. post- and pre-2008 crisis).

4.4 Impact on Transaction Costs

We follow the proportional transaction–cost framework of Ao, Li and Zheng (2019). All returns are *excess* returns, that is, expressed relative to a constant risk-free rate r_f.

Notation. Let N be the number of risky assets and T the number of monthly observations. For month t ($t = 1, \dots, T$) we denote

$$\mathbf{r}_t = \big(r_{1,t}, \dots, r_{N,t}\big)^\top \quad \text{excess returns,}$$
$$\mathbf{w}_t = \big(w_{1,t}, \dots, w_{N,t}\big)^\top \quad \text{portfolio weights } \textit{after} \text{ rebalancing at the } \textit{start} \text{ of month } t.$$

The portfolio is allowed to be leveraged or short, so $\sum_{i=1}^N w_{i,t}$ may differ from 1 and individual $w_{i,t}$ may be negative.
The implicit position in the risk-free asset is $w_{0,t} = 1 - \sum_{i=1}^N w_{i,t}$.

Step 1: Natural Drift. During month t the risky sleeve evolves with the realised returns, so at the *end* of the month the weights become

$$w_{i,t}^\star = \frac{w_{i,t}(1 + r_{i,t})}{w_{0,t}(1 + r_f) + \sum_{j=1}^N w_{j,t}(1 + r_{j,t})} \qquad (i = 1, \dots, N). \tag{3}$$

Equation (3) is identical to $w_j(t^+)$ in Ao, Li and Zheng (2019, Eq. 9).

Step 2: Target Weights. Using W_{est} months of data we compute new optimal weights $\mathbf{w}_{t+1}$ with our approach explained in 3.1 subject to the target volatility σ_{target}.

Step 3: Proportional Turnover Cost. The transaction to go from $\mathbf{w}_t^\star$ to the new target $\mathbf{w}_{t+1}$ incurs the proportional fee

$$\tau_{t+1} = \sum_{i=1}^N c_i \left| w_{i,t+1} - w_{i,t}^\star \right|, \tag{4}$$

which is precisely Eq. (6) in Ao, Li, and Zheng (2019).
In their research, the constants $c_i > 0$ are per-asset bid–ask spreads expressed as a fraction of notional value. For simplicity, we use the uniform fee $c_i = 10$ bp for all i in our empirical work.

Step 4: Net Portfolio Return. Let the *gross* portfolio excess return in month t+1 be $r_{t+1}^{\text{gross}} = \mathbf{w}_{t+1}^\top \mathbf{r}_{t+1}$. Transaction costs are paid once, exactly at the rebalance instant, so the *net* excess return is

$$r_{t+1}^{\text{net}} = (1 - \tau_{t+1})(1 + r_{t+1}^{\text{gross}}) - 1, \tag{5}$$

which matches Ao, Li and Zheng (2019, Eq. 10).[4]

[4]Multiplying by $(1 - \tau_{t+1})$ rather than subtracting τ_{t+1} from r_{t+1}^{gross} preserves the cross-term $\tau_{t+1} r_{t+1}^{\text{gross}}$, ensuring an exact net asset value (NAV) evolution.

Step 5: NAV Recursion. If V_t is the portfolio value at the start of month t, then

$$V_{t+1} = V_t\,(1 + r_{t+1}^{\text{net}}). \tag{6}$$

We iterate Steps 1–5 with rolling estimation and evaluation windows (W_{est}, W_{eval}) to obtain a full out-of-sample (OOS) path of net returns.

Discussion. Equation (4) penalises the *absolute* notional change, so exiting a short position and entering a long in the same asset is charged. Because weights are unconstrained, leverage enters the cost formula naturally: if $\sum_i w_{i,t+1} = 1.5$, then the net borrowing $w_{0,t+1} = -0.5$ is financed at the risk-free rate, but any change in that leverage is captured by (4) through the associated asset trades.

Equations (3)–(5) are implemented in our Python routine `rolling_backtest_tc` (Appendix I), ensuring an exact reproduction of the transaction-cost adjustments proposed by Ao, Li and Zheng (2019).

Table 4: Out-of-sample mean net volatility and Sharpe ratio under a proportional transaction cost of 10 bp per trade ($N = 112$).

Estimation Window (years)	**Volatility**		**Sharpe Ratio**	
	naive	**MAXSER**	**naive**	**MAXSER**
72(6)	0.232132	0.094227	0.030408	0.163781
84(7)	0.720208	0.085975	-0.023946	0.148963
96(8)	3.041235	0.082286	-0.199524	0.185639
108(9)	4.174943	0.082399	-0.370858	0.138392

Note. Transaction costs are already embedded in the reported net returns and hence in the volatility and Sharpe statistics. For reference, one can still report the cumulative turnover fee, $\sum_t \tau_t$, alongside these metrics.

Why transaction costs fall under MAXSER?

Figure 6 shows that the MAXSER allocation keeps only fifteen to forty active positions at any point in time, which is never more than one third of the investable universe. Fewer active weights imply smaller absolute rebalancing trades, so the turnover fee in (4) remains low. The lower trading bill explains why the net Sharpe ratio of MAXSER stays well above that of the sample–covariance benchmark even after the uniform ten-basis-point cost is deducted.

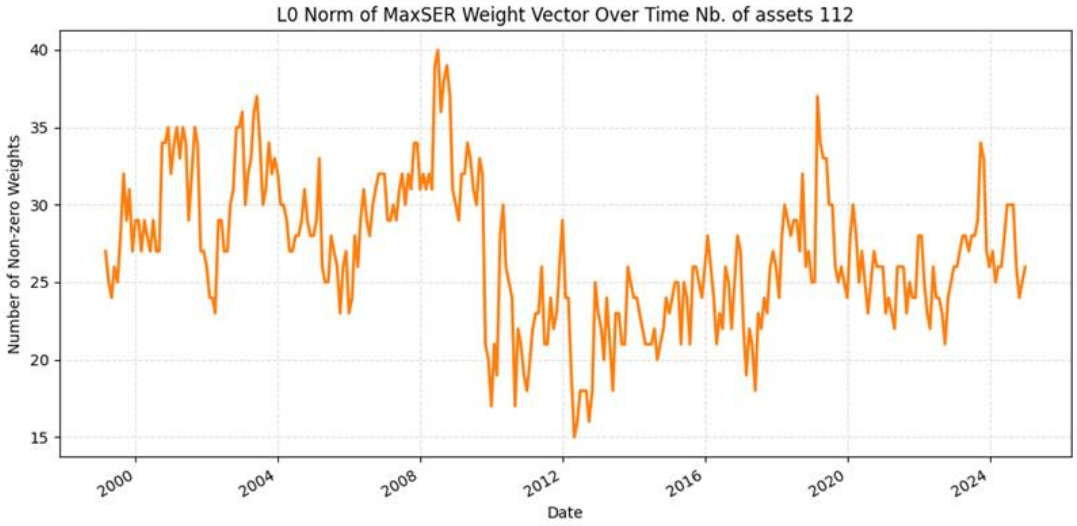


Figure 6: Time series of the ℓ_0 norm (number of non-zero weights) of the MAXSER portfolio for the $N = 112$ equity universe.

Across the analysis, three central lessons emerge, each is discussed in the conclusion below.

5 Conclusion

This research revisits the classical mean–variance framework in an environment where the number of risky assets rivals or exceeds the length of the available return history.
Two practical tools have been combined.

1. A **well conditioned covariance estimator** obtained from the linear shrinkage procedure of Ledoit and Wolf.

2. A **sparse volatility-budgeted optimiser** built on the Maximum Sharpe Estimated Regression (MAXSER) formulation, whose ℓ_1 penalty is chosen by a 10 fold cross validation.

Theoretical analysis shows that shrinkage guarantees the existence of a stable inverse covariance matrix and reduces the bias of the squared Sharpe statistic that enters MAXSER. Empirical evidence demonstrates that the two elements reinforce each other, producing portfolios that

- match the investor's volatility target,
- outperform a naive sample based Markowitz benchmark in net Sharpe ratio,
- cut turnover under a realistic trading cost of ten basis points per trade.

Main empirical lessons

- **Risk control.** Across different rolling windows the MAXSER portfolio remained close to the target risk while the naive often delivered two to fifty times the intended volatility.
- **Reward to risk.** After deducting trading costs, MAXSER retained a positive Sharpe advantage whenever the dimension ratio N/T exceeded one, a point where the plug-in solution became unreliable.
- **Economy of active positions.** Cross validation usually selected an ℓ_1 budget in the lowest decile of the LARS path. Suggesting that only a small subset of assets was needed to approach the efficient frontier once covariance noise had been shrunk.

Limitations

1. We used a spherical shrinkage target. A factor aligned or nonlinear target may further improve accuracy.

2. The optimisation considers only one period and remains myopic. Extending the method to a dynamic setting with multiple periods is reserved for future research.

3. A flat trading cost of ten basis points is an approximation. Liquidity sensitive impact models would offer finer realism.

4. Tests were limited to United States equities and Fama and French portfolios. Other asset classes may display different correlation structures and deserve separate investigation.

Future research

- Combine factor driven shrinkage with MAXSER in order to separate systematic and idiosyncratic risk more explicitly.

- Let the volatility budget itself evolve under a regime switching model that reflects changing risk appetite.

- Replace the deterministic ℓ_1 penalty by a hierarchical Bayesian prior, which would provide full posterior inference on the weights.

- Embed the entire pipeline in a live trading system that includes liquidity filters, tax lots and compliance constraints.

Final remark

More data and more assets should in principle enhance diversification, yet a naive application of the Markowitz equations often leads to unstable and costly allocations. By shrinking the covariance matrix, constraining the weights, and validating the volatility target on unseen data, this work shows that the efficient frontier can be approached once again in practice, not just in theory.

References

[1] Ao, M., Li, Y. and Zheng, X. (2019). *Approaching Mean–Variance Efficiency for Large Portfolios. Review of Financial Studies*, 32(7), 2890–2919.

[2] Ledoit, O. and Wolf, M. (2003). *Honey, I Shrunk the Sample Covariance Matrix.* Working Paper, Université de Genève.

[3] Ledoit, O. and Wolf, M. (2004). *A Well-Conditioned Estimator for Large-Dimensional Covariance Matrices. Journal of Multivariate Analysis*, 88(2), 365–411.

[4] Kan, R. and Zhou, G. (2007). *Optimal Portfolio Choice with Parameter Uncertainty. Journal of Financial and Quantitative Analysis*, 42(3), 621–656.

[5] Markowitz, H. (1952). *Portfolio Selection. Journal of Finance*, 7(1), 77–91.

[6] Efron, B., Hastie, T., Johnstone, I. and Tibshirani, R. (2004). *Least Angle Regression. Annals of Statistics*, 32(2), 407–499.

[7] Sharpe, W. F. (1966). *Mutual Fund Performance. Journal of Business*, 39(1), 119–138.

[8] Kenneth R. French Data Library. *Fama/French Data Library.* Available at: `https://mba.tuck.dartmouth.edu/pages/faculty/ken.french/data_library.html`.

[9] Wharton Research Data Services (WRDS). *WRDS Platform.* Available at: `https://wrds.wharton.upenn.edu/`.

6 Appendix

6.1 Appendix A: Derivation of Mean–Variance Weights with Target Return

We solve the classic Markowitz problem:

$$\min_{\mathbf{w}\in\mathbb{R}^n} \ \mathbf{w}^\top \Sigma \mathbf{w} \quad \text{s.t.} \quad \begin{cases} \mathbf{1}^\top \mathbf{w} = 1, \\ \boldsymbol{\mu}^\top \mathbf{w} = \mu_p. \end{cases} \tag{A.1}$$

Lagrangian Introduce multipliers λ and γ:

$$\mathcal{L}(\mathbf{w}, \lambda, \gamma) = \mathbf{w}^\top \Sigma \mathbf{w} - \lambda\big(\mathbf{1}^\top \mathbf{w} - 1\big) - \gamma\big(\boldsymbol{\mu}^\top \mathbf{w} - \mu_p\big).$$

First-order condition

$$2\Sigma\mathbf{w} - \lambda\mathbf{1} - \gamma\boldsymbol{\mu} = \mathbf{0} \implies \mathbf{w} = \tfrac{\lambda}{2}\Sigma^{-1}\mathbf{1} + \tfrac{\gamma}{2}\Sigma^{-1}\boldsymbol{\mu}.$$

Imposing constraints

$$A = \mathbf{1}^\top \Sigma^{-1}\mathbf{1}, \quad B = \mathbf{1}^\top \Sigma^{-1}\boldsymbol{\mu}, \quad C = \boldsymbol{\mu}^\top \Sigma^{-1}\boldsymbol{\mu}, \quad \Delta = AC - B^2 > 0.$$

$$\mathbf{1}^\top \mathbf{w} = \tfrac{\lambda}{2}A + \tfrac{\gamma}{2}B = 1, \qquad \boldsymbol{\mu}^\top \mathbf{w} = \tfrac{\lambda}{2}B + \tfrac{\gamma}{2}C = \mu_p.$$

$$\frac{\lambda}{2} = \frac{C - B\mu_p}{\Delta}, \qquad \frac{\gamma}{2} = \frac{A\mu_p - B}{\Delta}.$$

Closed-form weights

$$\boxed{\mathbf{w}^\star = \frac{C - B\mu_p}{\Delta}\,\Sigma^{-1}\mathbf{1} + \frac{A\mu_p - B}{\Delta}\,\Sigma^{-1}\boldsymbol{\mu}} \tag{A.2}$$

This is the unique mean–variance solution for target return μ_p.

6.2 Appendix B: Risk-target Mean–Variance Portfolio with Full Investment

We now derive the fully-invested risk-target formulation:

$$\max_{\mathbf{w}\in\mathbb{R}^n} \ \boldsymbol{\mu}^\top \mathbf{w} \quad \text{s.t.} \quad \begin{cases} \mathbf{1}^\top \mathbf{w} = 1, \\ \mathbf{w}^\top \Sigma \mathbf{w} = \sigma_{\text{target}}^2. \end{cases} \tag{B.1}$$

Lagrangian

$$\mathcal{L}(\mathbf{w}, \lambda, \gamma) = -\boldsymbol{\mu}^\top \mathbf{w} + \lambda\Big(\mathbf{w}^\top \Sigma \mathbf{w} - \sigma_{\text{target}}^2\Big) + \gamma\Big(\mathbf{1}^\top \mathbf{w} - 1\Big).$$

First-order condition

$$\boxed{\mathbf{w} = \frac{1}{2\lambda}\Sigma^{-1}\Big(\boldsymbol{\mu} - \gamma \mathbf{1}\Big)} \tag{B.2}$$

Notation

$$A = \mathbf{1}^\top \Sigma^{-1} \mathbf{1}, \quad B = \mathbf{1}^\top \Sigma^{-1} \boldsymbol{\mu}, \quad C = \boldsymbol{\mu}^\top \Sigma^{-1} \boldsymbol{\mu}, \quad \Delta = AC - B^2 > 0.$$

Basis representation

$$\mathbf{w} = a\,\Sigma^{-1}\mathbf{1} + b\,\Sigma^{-1}\boldsymbol{\mu}, \quad a = -\tfrac{\gamma}{2\lambda}, \quad b = \tfrac{1}{2\lambda}.$$

Constraints

$$aA + bB = 1, \qquad a^2 A + 2abB + b^2 C = \sigma_{\text{target}}^2.$$

Solution for a, b

$$\boxed{b = \frac{A - \sigma_{\text{target}}^2 B}{\sigma_{\text{target}}^2 \Delta}, \quad a = \frac{\sigma_{\text{target}}^2 C - B}{\sigma_{\text{target}}^2 \Delta}} \tag{B.3}$$

Optimal weights

$$\boxed{\mathbf{w}^*(\sigma_{\text{target}}) = \frac{\sigma_{\text{target}}^2 C - B}{\sigma_{\text{target}}^2 \Delta}\,\Sigma^{-1}\mathbf{1} + \frac{A - \sigma_{\text{target}}^2 B}{\sigma_{\text{target}}^2 \Delta}\,\Sigma^{-1}\boldsymbol{\mu}} \tag{B.4}$$

Expected return

$$\mu^*(\sigma_{\text{target}}) = \boldsymbol{\mu}^\top \mathbf{w}^* = \frac{1}{\sigma_{\text{target}}^2}\left(\sqrt{\Delta\Big(\sigma_{\text{target}}^2 - \sigma_{\text{GMV}}^2\Big)} + \tfrac{B}{A}\right), \quad \sigma_{\text{GMV}}^2 = \frac{1}{A}.$$

6.3 Appendix C: Efficient Portfolio with a Risk-Free Asset

Consider n risky assets whose excess-return vector is $\boldsymbol{r} \in \mathbb{R}^n$ with mean $\boldsymbol{\mu}$ and positive-definite covariance matrix Σ. Because a risk-free asset of *zero excess return* is available, no budget (full-investment) constraint is required on the risky-asset weights $\mathbf{w} \in \mathbb{R}^n$: any leftover wealth can be invested in the risk-free asset.

Fix a target volatility $\sigma > 0$ and set

$$\theta = \boldsymbol{\mu}^\top \Sigma^{-1} \boldsymbol{\mu}, \qquad r^* = \sigma\sqrt{\theta}.$$

$$\min_{\mathbf{w}\in\mathbb{R}^n} \ \mathbf{w}^\top \Sigma \mathbf{w} \quad \text{s.t.} \quad \boldsymbol{\mu}^\top \mathbf{w} \geq r^*. \tag{C.1}$$

Why the inequality binds. If a feasible $\tilde{\mathbf{w}}$ satisfies $\boldsymbol{\mu}^\top \tilde{\mathbf{w}} > r^*$, scaling by $c = r^*/\left(\boldsymbol{\mu}^\top \tilde{\mathbf{w}}\right) \in (0,1)$ produces $c\tilde{\mathbf{w}}$ with return r^* and strictly smaller variance. Hence the optimum of (C.1) attains the *equality* $\boldsymbol{\mu}^\top \mathbf{w} = r^*$.

Lagrangian. With the constraint written as an equality,

$$\mathcal{L}(\mathbf{w}, \lambda) = \mathbf{w}^\top \Sigma \mathbf{w} - \lambda\left(\boldsymbol{\mu}^\top \mathbf{w} - r^*\right).$$

Stationarity (gradient condition).

$$\boxed{\nabla_{\mathbf{w}} \mathcal{L} = 2\Sigma\mathbf{w} - \lambda\boldsymbol{\mu} = \mathbf{0}} \quad \Longrightarrow \quad \mathbf{w} = \tfrac{\lambda}{2}\,\Sigma^{-1}\boldsymbol{\mu}. \tag{C.2}$$

Solving for λ. Substitute (C.2) into the binding return constraint:

$$\frac{\lambda}{2}\,\theta = r^* \implies \boxed{\lambda = 2\,\frac{r^*}{\theta} = 2\,\frac{\sigma}{\sqrt{\theta}}}. \tag{C.3}$$

Optimal risky-asset weights. Inserting λ from (C.3) into (C.2) gives

$$\boxed{\mathbf{w}^* = \frac{\sigma}{\sqrt{\theta}}\,\Sigma^{-1}\boldsymbol{\mu}}. \tag{C.4}$$

Verification of the risk target.

$$(\mathbf{w}^*)^\top \Sigma \mathbf{w}^* = \left(\tfrac{\sigma}{\sqrt{\theta}}\right)^2 \boldsymbol{\mu}^\top \Sigma^{-1} \boldsymbol{\mu} = \sigma^2,$$

so the target volatility is met.

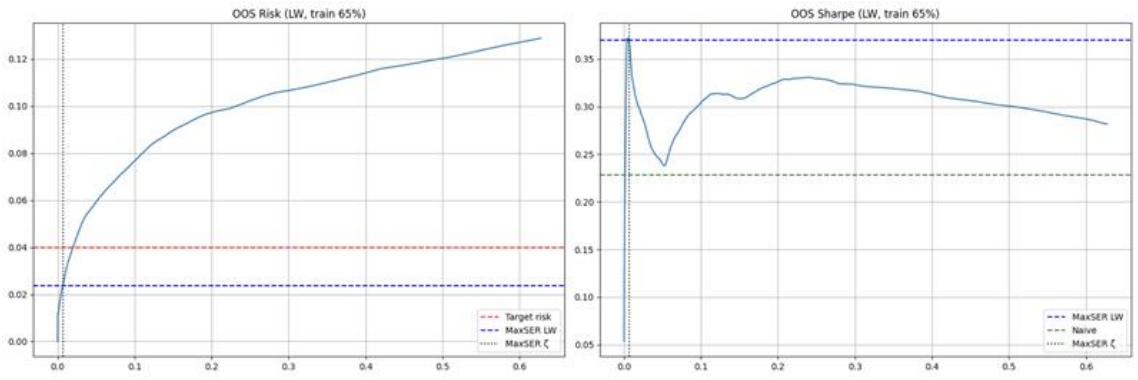
Risk-free allocation. Normalising wealth to 1, allocate

$$w_f^* \;=\; 1 - \mathbf{1}^\top \mathbf{w}^*,$$

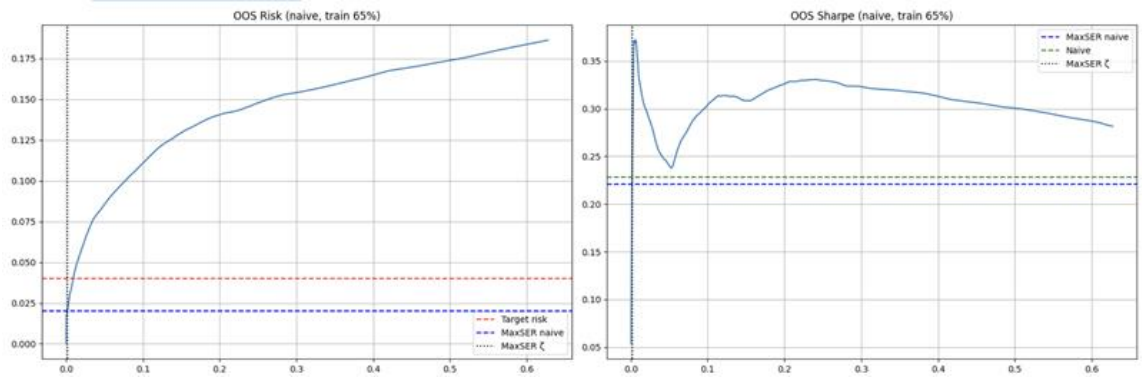
where $\mathbf{1}$ is the n-vector of ones. Varying σ traces the Capital Market Line through the maximum-Sharpe portfolio.

Summary. With a risk-free asset, the efficient risky allocation is always a scalar multiple of $\Sigma^{-1}\boldsymbol{\mu}$. Scaling by $\sigma/\sqrt{\theta}$ achieves any desired volatility while preserving the maximum Sharpe ratio $\sqrt{\theta}$, so the inequality in (C.1) binds at the optimum.

6.4 Appendix D: Extended Out-of-Sample Comparison (65 % Training Window)



(a) Ledoit–Wolf covariance, training share 65 %



(b) Sample covariance, training share 65 %

Figure 7: Out-of-sample risk (left panels) and Sharpe ratio (right panels) along the MAXSER LARS path when sixty-five percent of the data are used for estimation.

6.5 Appendix E: Additional Rolling Windows (2000–2024)

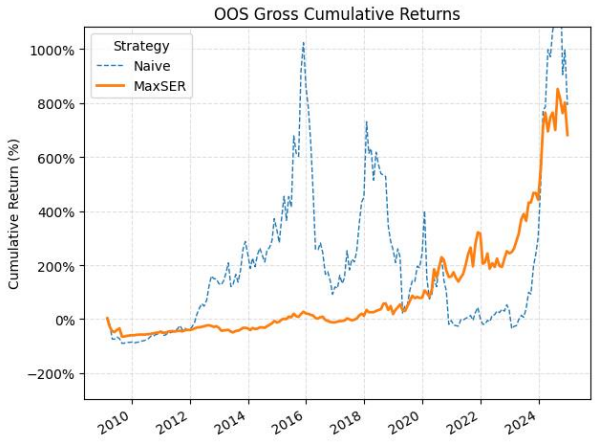


Figure 8: Out-of-sample cumulative returns (2000–2024) with a 108-month window.

Figure 9: Monthly volatility across time (2000–2024) with a 108-month window.

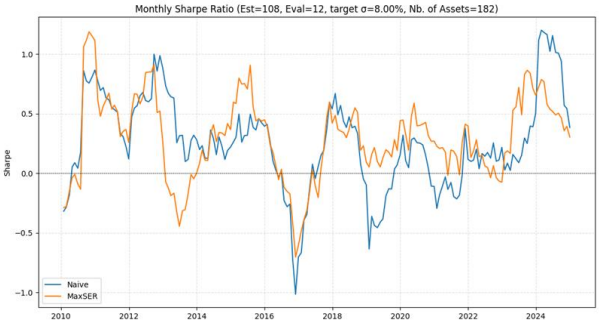


Figure 10: Monthly Sharpe ratio across time (2000–2024) with a 108-month window.

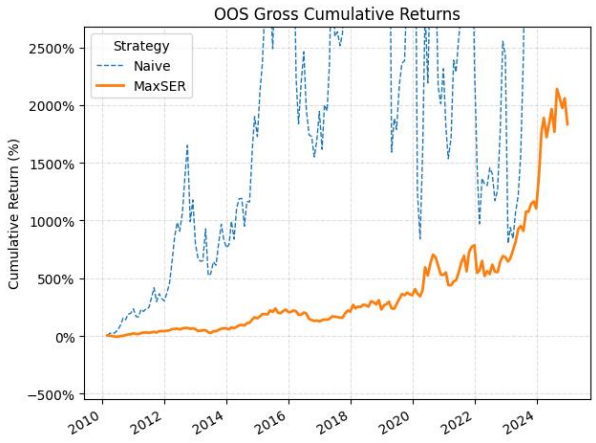


Figure 11: Out-of-sample cumulative returns (2000–2024) with a 120-month window.

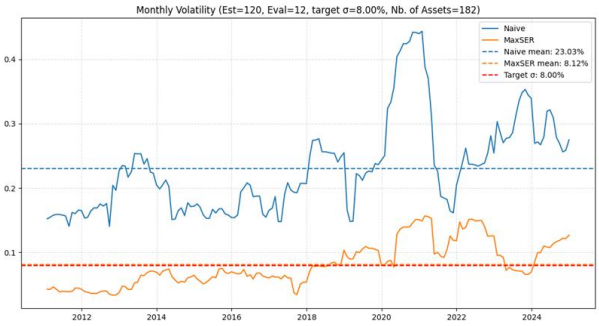


Figure 12: Monthly volatility across time (2000–2024) with a 120-month window.

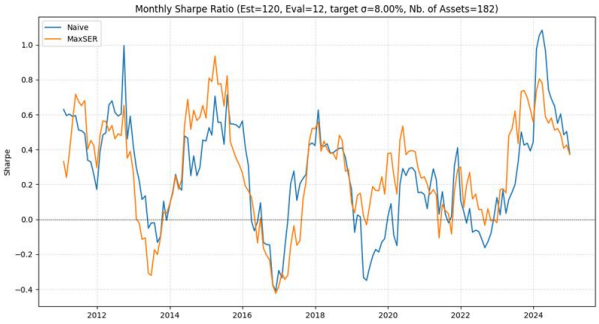


Figure 13: Monthly Sharpe ratio across time (2000–2024) with a 120-month window.

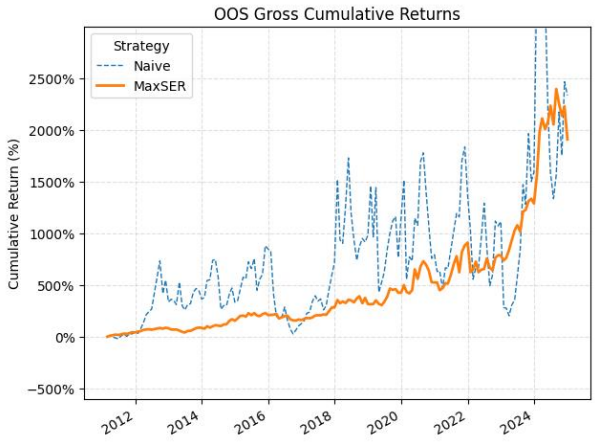


Figure 14: Out-of-sample cumulative returns (2000–2024) with a 132-month window.

Figure 15: Monthly volatility across time (2000–2024) with a 132-month window.

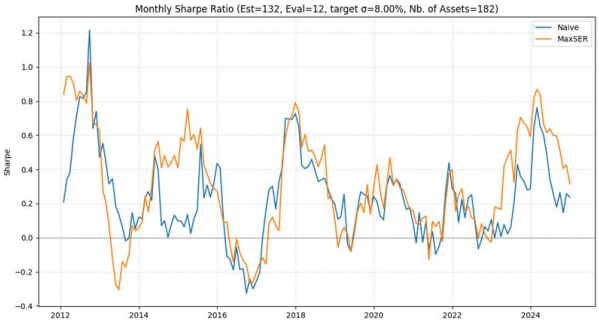


Figure 16: Monthly Sharpe ratio across time (2000–2024) with a 132-month window.

6.6 Appendix F: Additional Rolling Windows (1990–2024)

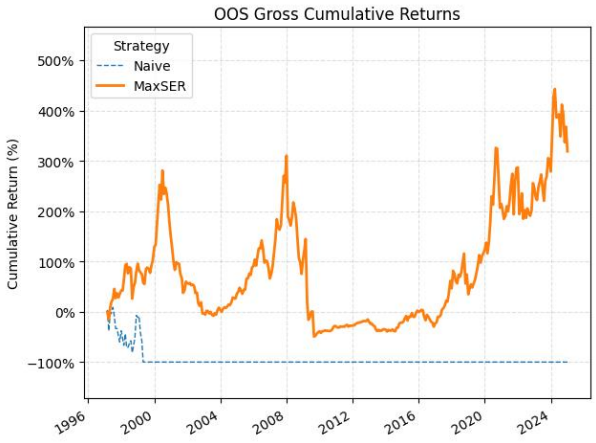


Figure 17: Out-of-sample cumulative returns (1990–2024) with a 84-month window.

Figure 18: Monthly volatility across time (1990–2024) with a 84-month window.

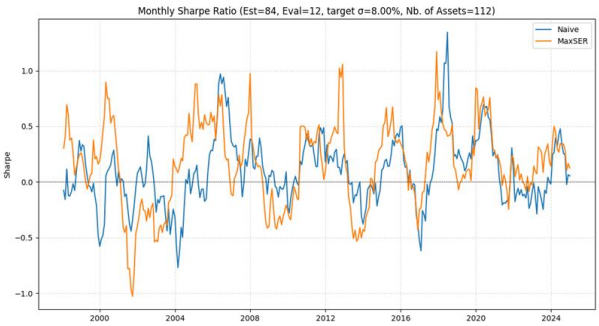


Figure 19: Monthly Sharpe ratio across time (1990–2024) with a 84-month window.

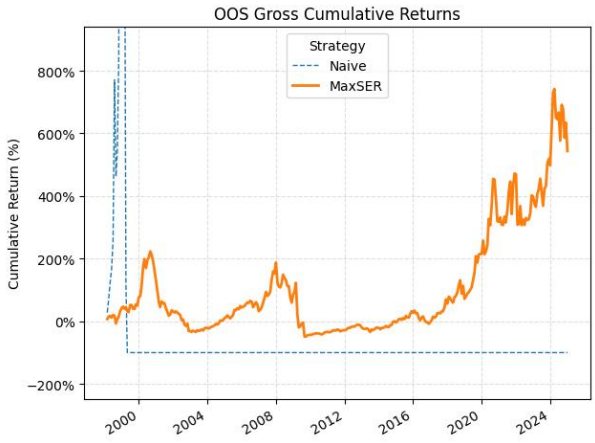


Figure 20: Out-of-sample cumulative returns (1990–2024) with a 96-month window.

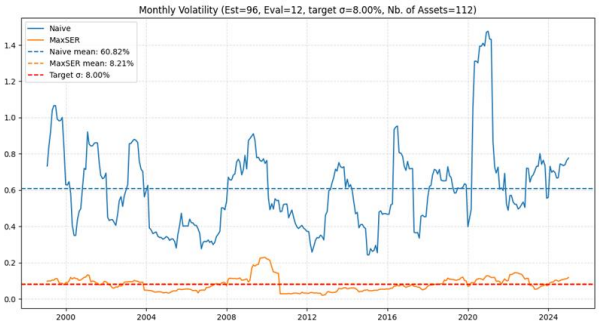


Figure 21: Monthly volatility across time (1990–2024) with a 96-month window.

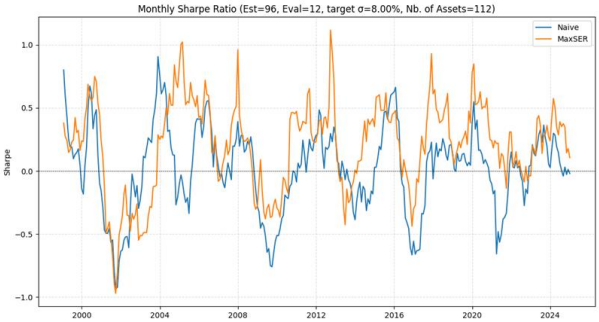


Figure 22: Monthly Sharpe ratio across time (1990–2024) with a 96-month window.

6.7 Appendix G: `solutionpath.py`

```python
# solutionpath.py
# Authors: Tristan Gonçalves & Pierre-François Pinelli

import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pandas_datareader import data
from sklearn.linear_model import lars_path
from sklearn.covariance import ledoit_wolf
from sklearn.model_selection import KFold
import ipywidgets as widgets
from IPython.display import display
warnings.simplefilter(action="ignore", category=FutureWarning)
warnings.filterwarnings("ignore", category=RuntimeWarning)
np.seterr(over="ignore", divide="ignore", invalid="ignore", under="ignore")

#
↪ ============================================================================
# 1. Data Loading and Preparation
#
↪ ============================================================================
start_date = "1980-01-01"
end_date   = "2024-12-31"

ff_queries = [
    "Portfolios_Formed_on_BE-ME",
    "48_Industry_Portfolios",
    "Portfolios_Formed_on_INV",
    "100_Portfolios_10x10",
    "100_Portfolios_ME_OP_10x10",
]
frames = [data.DataReader(q, "famafrench", start_date, end_date)[0] for q in
↪ ff_queries]

df = pd.concat(frames, axis=1).dropna()
print(f"Combined data shape: {df.shape}")

rf_monthly = (1 + 0.02) ** (1/12) - 1
R = df.values / 100 - rf_monthly  # excess returns
```

```
38
39  #
    ↪  ============================================================================
40  # 2. Helper Functions
41  #
    ↪  ============================================================================
42
43  def covariance_matrix(X: np.ndarray, method: str = "naive") -> np.ndarray:
44      if method == "naive":
45          return np.cov(X, rowvar=False)
46      elif method == "LW":
47          return ledoit_wolf(X)[0]
48      raise ValueError("method  {naive, LW}")
49
50  def compute_theta_hat(X: np.ndarray, method: str = "naive") -> float:
51      mu = X.mean(axis=0)
52      Sigma = covariance_matrix(X, method)
53      return float(mu @ np.linalg.pinv(Sigma) @ mu)
54
55  def portfolio_naive(X: np.ndarray, sigma_target: float) -> np.ndarray:
56      theta = compute_theta_hat(X, "naive")
57      mu    = X.mean(axis=0)
58      invS  = np.linalg.pinv(covariance_matrix(X, "naive"))
59      return (sigma_target / np.sqrt(theta)) * (invS @ mu)
60
61  def _ols_solution(A: np.ndarray, b: np.ndarray) -> np.ndarray:  # helper
62      return np.linalg.pinv(A) @ b
63
64  def _l1_norm(w: np.ndarray) -> float:
65      return float(np.sum(np.abs(w)))
66
67  def _portfolio_risk(w: np.ndarray, S: np.ndarray) -> float:
68      return float(np.sqrt(w @ S @ w))
69
70  def portfolio_maxser(
71      X: np.ndarray,
72      sigma_target: float,
73      cov_method: str = "naive",
74      n_splits: int = 10,
75  ) -> np.ndarray:
76      """Max-SER portfolio for chosen covariance estimator."""
```

```
    T, _ = X.shape
    S_full = covariance_matrix(X, cov_method)
    mu     = X.mean(axis=0)
    theta  = float(mu @ np.linalg.pinv(S_full) @ mu)
    r_c    = sigma_target * (1 + theta) / np.sqrt(theta)
    y_full = np.full(T, r_c)

    # cross-validated
    kf = KFold(n_splits=n_splits)
    lambda_list = []
    for tr_idx, val_idx in kf.split(X):
        X_tr, X_val = X[tr_idx], X[val_idx]
        y_tr = np.full(len(tr_idx), r_c)
        w_ols = _ols_solution(X_tr, y_tr)
        norm_ols = _l1_norm(w_ols)

        alphas, _, coefs = lars_path(X_tr, y_tr, method="lasso")

        gaps = np.abs([
            _portfolio_risk(coefs[:, i], covariance_matrix(X_val, cov_method))
            ↪ - sigma_target
            for i in range(coefs.shape[1])
        ])
        lambda_list.append(alphas[int(np.argmin(gaps))])

    lambda_hat = float(np.mean(lambda_list))

    alpha, _, coefs_full = lars_path(X, y_full, method="lasso")
    idx_final = int(np.argmin(np.abs(alpha - lambda_hat)))
    return coefs_full[:, idx_final]

def solution_path(X: np.ndarray, sigma_target: float, cov_method: str =
↪ "naive"):
    theta = compute_theta_hat(X, cov_method)
    r_c   = sigma_target * (1 + theta) / np.sqrt(theta)
    y     = np.full(X.shape[0], r_c)
    _, _, coefs = lars_path(X, y, method="lasso", Gram="auto")
    risks = np.std(X @ coefs, axis=0)
    return risks, coefs
```

```
#
↪ ================================================================================
# 3. Quick In-Sample Comparison
#
↪ ================================================================================

sigma = 0.04
w_naive        = portfolio_naive(R, sigma)
w_maxser_naive = portfolio_maxser(R, sigma, "naive")
w_maxser_LW    = portfolio_maxser(R, sigma, "LW")

print("\n=== w (in-sample) ===")
print("Naive:       ", np.sum(w_naive))
print("MaxSER naïve:", np.sum(w_maxser_naive))
print("MaxSER LW:   ", np.sum(w_maxser_LW))

#
↪ ================================================================================
# 4. Comparison Table Function
#
↪ ================================================================================

def compare_methods(train_fractions=None, sigma_target: float = sigma):
    """Return a DataFrame with risk & Sharpe for each method × train size."""
    if train_fractions is None:
        train_fractions = np.arange(0.2, 0.9, 0.1)

    records = []
    for tf in train_fractions:
        n_train = int(tf * len(R))
        R_tr, R_te = R[:n_train], R[n_train:]
        mu_te = R_te.mean(axis=0)

        # Naive simple
        w_nv = portfolio_naive(R_tr, sigma_target)
        risk_nv = np.std(R_te @ w_nv)
        sharpe_nv = (mu_te @ w_nv) / risk_nv
        records.append(dict(train=tf, method="Naive",   risk=risk_nv,
        ↪  sharpe=sharpe_nv))

        # MaxSER naïve
```

```
151         w_ms_nv = portfolio_maxser(R_tr, sigma_target, "naive")
152         risk_ms_nv = np.std(R_te @ w_ms_nv)
153         sharpe_ms_nv = (mu_te @ w_ms_nv) / risk_ms_nv
154         records.append(dict(train=tf, method="MaxSER_naive", risk=risk_ms_nv,
            ↪  sharpe=sharpe_ms_nv))
155
156         # MaxSER LW
157         w_ms_lw = portfolio_maxser(R_tr, sigma_target, "LW")
158         risk_ms_lw = np.std(R_te @ w_ms_lw)
159         sharpe_ms_lw = (mu_te @ w_ms_lw) / risk_ms_lw
160         records.append(dict(train=tf, method="MaxSER_LW", risk=risk_ms_lw,
            ↪  sharpe=sharpe_ms_lw))
161
162     df_cmp = pd.DataFrame(records)
163     return df_cmp.pivot(index="train", columns="method", values=["risk",
        ↪  "sharpe"]).sort_index()
164
165 #
    ↪  ============================================================================
166 # 5. Interactive Plot (unchanged but with dropdown)
167 #
    ↪  ============================================================================
168
169 def plot_for_train_fraction(train_fraction: float, cov_method: str):
170     n_train = int(train_fraction * len(R))
171     R_tr, R_te = R[:n_train], R[n_train:]
172     mu_te = R_te.mean(axis=0)
173
174     theta_tr = compute_theta_hat(R_tr, cov_method)
175     r_c_tr = sigma * (1 + theta_tr) / np.sqrt(theta_tr)
176     y_tr = np.full(R_tr.shape[0], r_c_tr)
177     ols_tr = np.linalg.lstsq(R_tr, y_tr, rcond=None)[0]
178     norm_ols = _l1_norm(ols_tr)
179
180     _, _, coefs_tr = lars_path(R_tr, y_tr, method="lasso", Gram="auto")
181     zeta_tr = np.sum(np.abs(coefs_tr), axis=0) / norm_ols
182
183     w_ms = portfolio_maxser(R_tr, sigma, cov_method)
184     r_ms = np.std(R_te @ w_ms)
185     z_ms = _l1_norm(w_ms) / norm_ols
186     s_ms = (mu_te @ w_ms) / r_ms
```

```
187
188     w_nv = portfolio_naive(R_tr, sigma)
189     r_nv = np.std(R_te @ w_nv)
190     s_nv = (mu_te @ w_nv) / r_nv
191
192     risks = [np.std(R_te @ w) for w in coefs_tr.T]
193     sharpes = [(mu_te @ w) / r for w, r in zip(coefs_tr.T, risks)]
194
195     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, 6))
196
197     ax1.plot(zeta_tr, risks)
198     ax1.axhline(sigma, color="r", linestyle="--", label="Target risk")
199     ax1.axhline(r_ms, color="b", linestyle="--", label=f"MaxSER {cov_method}")
200     ax1.axvline(z_ms, color="k", linestyle=":", label="MaxSER ")
201     ax1.set_title(f"OOS Risk ({cov_method}, train {train_fraction:.0%})")
202     ax1.legend(); ax1.grid()
203
204     ax2.plot(zeta_tr, sharpes)
205     ax2.axhline(s_ms, color="b", linestyle="--", label=f"MaxSER {cov_method}")
206     ax2.axhline(s_nv, color="g", linestyle="--", label="Naive")
207     ax2.axvline(z_ms, color="k", linestyle=":", label="MaxSER ")
208     ax2.set_title(f"OOS Sharpe ({cov_method}, train {train_fraction:.0%})")
209     ax2.legend(); ax2.grid()
210
211     plt.tight_layout()
212     plt.show()
213
214 slider = widgets.FloatSlider(value=0.6, min=0.1, max=0.9, step=0.05,
215                              description="Train size:", readout_format=".0%")
216 method_dropdown = widgets.Dropdown(options=[("Naive", "naive"), ("Ledoit-Wolf",
    ↪  "LW")],
217                                    value="naive", description="Covariance:")
218
219 interactive_plot = widgets.interactive(plot_for_train_fraction,
220                                        train_fraction=slider,
221                                        cov_method=method_dropdown)
222
223 def run_interactive():
224     display(interactive_plot)
225
```

```
226 #
    ↪ ================================================================================
227 # 6. Run
228 #
    ↪ ================================================================================
229 if __name__ == "__main__":
230     tbl = compare_methods()
231     print("\n=== Comparison table (risk / Sharpe) ===")
232     print(tbl.round(4))
233 
234     print("\nMove the slider / dropdown for interactive plots ...")
235     run_interactive()
```

6.8 Appendix H: rollingwindow.py

```python
#rollingwindow.py
#Authors: Tristan Gonçalves & Pierre-François Pinelli
#%%

# 0. USER PARAMETERS
WRDS_USER = "trigo06"
START     = "1990-01-31"
END       = "2024-12-31"
SIGMA_TGT = 0.08                  # target monthly volatility
WIN_EST   = 12*6                 # estimation window (months)
WIN_EVAL  = 12                  # evaluation window (months)
RF_ANNUAL = 0.02                  # 2% per year
RF_MONTH  = (1 + RF_ANNUAL)**(1/12) - 1
#%%
all_tickers = [
    "A", "AAPL", "ABT", "ADBE", "ADI", "ADM", "ADSK", "AEE", "AEP", "AES",
    "AFL", "AIG", "AJG", "AKAM", "ALB", "ALL", "AMAT", "AMD", "AME", "AMGN",
    "AMT", "AMZN", "ANSS", "AOS", "APA", "APD", "APH", "ARE", "ATO", "AVB",
    ↪  "AVY",
    "AXP", "AZO", "BA", "BAC", "BAX", "BBY", "BDX", "BK", "BLK", "BMY", "BRO",
    "BSX", "BXP", "C", "CAG", "CAH", "CAT", "CB", "CCL", "CHD", "CHRW",
    "CI", "CINF", "CL", "CLX", "CMS", "COF", "COO", "COST", "CPB", "CPRT",
    "CPT", "CSCO", "CSGP", "CSX", "CTAS", "CTSH", "CVS", "D", "DE", "DHI",
    "DHR", "DLTR", "DOV", "DVN", "EBAY", "ECL", "ED", "EOG", "ETN", "ETR",
    "EXPD", "F", "FCX", "FITB", "FMC", "GD", "GE", "GILD", "GIS", "GLW",
    "HAL", "HBAN", "HD", "HIG", "HOLX", "HON", "HRL", "HSY", "HUM", "IBM",
    "INTC", "INTU", "IP", "IPG", "IRM", "IT", "ITW", "JBHT", "JCI", "JNJ",
    "JNPR", "JPM", "K", "KEY", "KIM", "KLAC", "KMB", "KO", "KR", "LLY",
    "LMT", "LNT", "LRCX", "LUV", "MAA", "MAR", "MCD", "MCHP", "MCK", "MDT",
    "MLM", "MMC", "MMM", "MO", "MRK", "MSFT", "MTB", "MTD", "MU",
    "NEM", "NKE", "NOC", "NSC", "NTAP", "NTRS", "NUE", "NVDA", "NVR",
    "NWL", "O", "ODFL", "OKE", "OMC", "ORCL", "ORLY", "OXY", "PAYX",
    "PCAR", "PCG", "PEG", "PEP", "PFE", "PG", "PGR", "PH", "PHM", "PLD",
    "PNC", "PNW", "PPG", "PSA", "PWR", "QCOM", "RCL", "REG", "REGN", "RHI",
    "RJF", "RL", "TECH"
]
#%%
# 1. IMPORTS
import wrds
import numpy as np
```

```
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import chi2
from sklearn.linear_model import lars_path
from sklearn.covariance import ledoit_wolf
import matplotlib.ticker as mtick
import warnings
from sklearn.model_selection import TimeSeriesSplit
from sklearn.model_selection import KFold
warnings.simplefilter(action='ignore', category=FutureWarning)
warnings.filterwarnings("ignore", category=RuntimeWarning)
np.seterr(over="ignore", divide="ignore", invalid="ignore", under="ignore")
#%%
# 2. DATA ACQUISITION FROM WRDS
print("Connecting to WRDS...")
db = wrds.Connection(wrds_username=WRDS_USER)

tickers_sql = ",".join(f"'{t}'" for t in all_tickers)
query = f"""
    SELECT m.date, s.ticker, m.ret
    FROM   crsp.msf m
    JOIN   crsp.msenames s
      ON   m.permno = s.permno
     AND   m.date BETWEEN s.namedt AND s.nameendt
    WHERE  m.date BETWEEN '{START}' AND '{END}'
      AND  s.ticker IN ({tickers_sql})
    ORDER  BY m.date, s.ticker
"""
df = db.raw_sql(query)
db.close()

df['date'] = pd.to_datetime(df['date'])
df = df.drop_duplicates(subset=['date', 'ticker'], keep='last')
df_pivot = df.pivot(index='date', columns='ticker', values='ret')

# drop tickers with missing or zero returns
df_clean = df_pivot.dropna(axis=1)
df_clean = df_clean.loc[:, (df_clean != 0).any(axis=0)]
print(f"Retained {df_clean.shape[1]} tickers over {df_clean.shape[0]}
↪  months.")

```

```
R_full = df_clean.values.astype(float) - RF_MONTH
dates  = df_clean.index
T, N   = R_full.shape
#%%
# 3. PORTFOLIO FUNCTIONS

def CovarianceMatrix(R, method="naive"):
    """Return covariance matrix of returns R."""
    if method == "naive":
        return np.cov(R, rowvar=False)
    elif method == "LW":
        return ledoit_wolf(R)[0]
    else:
        raise ValueError("Unknown covariance method.")

def compute_theta(R, method="naive"):
    """Compute theta = mu' Sigma_hat_inv mu."""
    mu = R.mean(axis=0)
    Sigma_hat  = CovarianceMatrix(R, method)
    return mu @ np.linalg.pinv(Sigma_hat) @ mu

def portfolio_naive(R, sigma):
    """Naive mean-variance weight vector (scaled to target vol)."""
    mu      = R.mean(axis=0)
    Sigma_hat       = CovarianceMatrix(R, "naive")
    theta   = compute_theta(R, "naive")
    inv_cov = np.linalg.pinv(Sigma_hat)
    return (sigma / np.sqrt(theta)) * (inv_cov @ mu)

def _ols(X, y):
    return np.linalg.pinv(X) @ y

def _l1_norm(w):
    return np.sum(np.abs(w))

def _volatility(w, Sigma_hat):
    return np.sqrt(w @ Sigma_hat @ w)

def portfolio_maxser(R, sigma_target, n_folds=10):
    """
    MaxSER: find lasso regression weights
```

```
    that match target volatility on average.
    """
    T, _ = R.shape
    Sigma_hat_full = ledoit_wolf(R)[0]
    mu     = R.mean(axis=0)
    theta_hat      = mu @ np.linalg.pinv(Sigma_hat_full) @ mu
    r_c    = sigma_target * (1 + theta_hat) / np.sqrt(theta_hat)
    y_full = np.full(T, r_c)

    # Cross-validate lambda target using K-fold
    kf, lambdas = KFold(n_splits=n_folds), []
    for tr_idx, val_idx in kf.split(R):
        X_tr, X_val = R[tr_idx], R[val_idx]
        y_tr        = np.full(len(tr_idx), r_c)

        # lasso path
        alphas, _, coefs = lars_path(X_tr, y_tr, method="lasso")
        # pick lambda that yields closest vol on hold-out
        gaps = np.abs([
            _volatility(w, ledoit_wolf(X_val)[0]) - sigma_target
            for w in coefs.T
        ])
        lambdas.append(alphas[np.argmin(gaps)])

    lambda_hat = np.mean(lambdas)

    # final fit on full sample
    alpha_full, _, coefs_full = lars_path(R, y_full, method="lasso")
    idx_final       = np.argmin(np.abs(alpha_full - lambda_hat))
    return coefs_full[:, idx_final]

# 4. ROLLING BACKTEST

HOLD = 1   # how many future months you keep the weights unchanged

def rolling_backtest(R, dates, window_est, hold, sigma, strategy_fn):
    """
    * window_est: number of past months used for parameter estimation.
    * hold:       number of months the resulting weights are kept unchanged.
    Returns OOS monthly returns, the corresponding weight vectors and their
    ↪ dates.
```

```
    """
    rets, wts, idx = [], [], []
    t = window_est
    while t + hold <= len(R) - 1:
        w = strategy_fn(R[t - window_est : t], sigma)  # rebalance
        for h in range(1, hold + 1):                   # hold period
            rets.append(R[t + h] @ w)
            wts.append(w)                              # store w every month
            idx.append(dates[t + h])
        t += hold
    return np.array(rets), wts, pd.DatetimeIndex(idx)


#%%
# 6. RUN BACKTESTS
# ----------------------------------------------------------------
ret_n, w_n, idx_n = rolling_backtest(R_full, dates,
                                     WIN_EST, HOLD, SIGMA_TGT,
                                     portfolio_naive)
ret_m, w_m, idx_m = rolling_backtest(R_full, dates,
                                     WIN_EST, HOLD, SIGMA_TGT,
                                     portfolio_maxser)


df_ret = pd.DataFrame({
    "Naive": ret_n,
    "MaxSER": ret_m
}, index=idx_n)

# 6. OUT-OF-SAMPLE METRICS
vol_oos = df_ret.rolling(WIN_EVAL).std(ddof=1)
shp_oos = df_ret.rolling(WIN_EVAL).apply(lambda x: x.mean()/x.std(ddof=1),
    raw=True)

# 95% CI for volatility based on ² (with strong assumption normality of
    returns)
alpha = 0.05
df_    = WIN_EVAL - 1
var_oos    = df_ret.rolling(WIN_EVAL).var(ddof=1)
chi2_low   = chi2.ppf(1 - alpha/2, df_)
chi2_high  = chi2.ppf(alpha/2, df_)
```

```
var_low    = (df_ * var_oos) / chi2_low
var_high   = (df_ * var_oos) / chi2_high
vol_low    = np.sqrt(var_low)
vol_high   = np.sqrt(var_high)

# 7. PLOTTING
#%%
# Cumulative returns
plt.figure(figsize=(12, 6))

cum = df_ret.add(1).cumprod() - 1                # decimal form
cum_floor = cum.mask((cum <= -1).cummax(), -1)
cum_pct = cum_floor * 100                        # convert to %

# emphasize MaxSER
ax = cum_pct.plot(style=['--', '-'])
ax.lines[0].set_linewidth(1)
ax.lines[1].set_linewidth(2)

plt.title("OOS Gross Cumulative Returns")
plt.ylabel("Cumulative Return (%)")

# centre y-axis on MaxSER
maxser = cum_pct["MaxSER"]
mid  = (maxser.max() + maxser.min()) / 2
span = (maxser.max() - maxser.min()) * 1.5
ax.set_ylim(mid - span/2, mid + span/2)

plt.gca().yaxis.set_major_formatter(mtick.PercentFormatter())
plt.grid(ls='--', alpha=0.4)
plt.legend(title="Strategy")
plt.tight_layout()
plt.show()

# Volatility
plt.figure(figsize=(11, 6))

mean_naive  = vol_oos['Naive'].mean()
mean_maxser = vol_oos['MaxSER'].mean()

plt.plot(vol_oos.index, vol_oos['Naive'],   color='C0', label='Naive')
```

```
238 plt.plot(vol_oos.index, vol_oos['MaxSER'],  color='C1', label='MaxSER')
239
240 plt.axhline(mean_naive,  ls='--', color='C0',
241             label=f'Naive mean: {mean_naive:.2%}')
242 plt.axhline(mean_maxser, ls='--', color='C1',
243             label=f'MaxSER mean: {mean_maxser:.2%}')
244 plt.axhline(SIGMA_TGT,   ls='--', color='r',
245             label=f'Target Sigma_hat: {SIGMA_TGT:.2%}')
246
247 plt.title(
248     f"Monthly Volatility (Est={WIN_EST}, Eval={WIN_EVAL}, "
249     f"target Sigma_hat={SIGMA_TGT:.2%}, Nb. of Assets={N})"
250 )
251 plt.grid(ls='--', alpha=0.4)
252 plt.tight_layout()
253 plt.legend()
254 plt.show()
255
256 # Sharpe Ratio
257 plt.figure(figsize=(11, 6))
258 plt.plot(shp_oos.index, shp_oos['Naive'], label='Naive')
259 plt.plot(shp_oos.index, shp_oos['MaxSER'], label='MaxSER')
260 plt.axhline(0, ls='--', color='k', lw=0.5)
261 plt.title(f"Monthly Sharpe Ratio (Est={WIN_EST}, Eval={WIN_EVAL}, target
    ↪  Sigma_hat={SIGMA_TGT:.2%}, Nb. of Assets={N})")
262 plt.ylabel("Sharpe")
263 plt.legend()
264 plt.grid(ls='--', alpha=0.4)
265 plt.tight_layout()
266 plt.show()
267
268 # 8. SUMMARY
269 print("\n--- Mean OOS Metrics ---")
270 print("Volatility:\n",vol_oos.mean())
271 print("Sharpe:\n",shp_oos.mean())
272
273 np.mean(ret_m)/np.std(ret_m)
274 np.mean(ret_m)/np.std(ret_m)
275 mu_m = np.mean(ret_m)
276 mu_n = np.mean(ret_n)
277 print(f"MaxSER expected return: {mu_m:.4f}")
```

```python
print(f"Naive expected return: {mu_n:.4f}")
std_maxser = np.std(ret_m)
std_naive  = np.std(ret_n)
print(f"MaxSER Std: {std_maxser:.4f}")
print(f"Naive Std: {std_naive:.4f}")
print(f"MaxSER Sharpe: {np.mean(ret_m)/std_maxser:.4f}")
print(f"Naive Sharpe: {np.mean(ret_n)/std_naive:.4f}")



# %%
# Compute and plot the L0-norm (number of non-zero weights) of MaxSER over time
l0_maxser = [np.count_nonzero(w) for w in w_m]
l0_series = pd.Series(l0_maxser, index=idx_m)

plt.figure(figsize=(10, 5))
l0_series.plot(color='C1', linewidth=2)
plt.title(f"L0 Norm of MaxSER Weight Vector Over Time Nb. of assets {N}")
plt.ylabel('Number of Non-zero Weights')
plt.xlabel('Date')
plt.grid(ls='--', alpha=0.4)
plt.tight_layout()
plt.show()
# %%

```

6.9 Appendix I: `rollingwindow_tc.py`

```python
# rollingwindow_tc.py
# Authors: Tristan Gonçalves & Pierre-François Pinelli
#%%
# 0. USER PARAMETERS
# ------------------------------------------------------------
WRDS_USER = "trigo06"
START     = "1990-01-31"
END       = "2024-12-31"
SIGMA_TGT = 0.08                # target monthly volatility
WIN_EST   = 12*9                # estimation window (months)
WIN_EVAL  = 12                    # evaluation window (months)
TC_RATE   = 0.001                # 0.10% proportional fee per trade (10bps)
RF_ANNUAL = 0.02                 # risk-free, used only for excess returns
RF_MONTH  = (1 + RF_ANNUAL)**(1/12) - 1
HOLD      = 1                    # rebalance & hold horizon (months)
#%%
# 1. IMPORTS
# ------------------------------------------------------------------------
import wrds
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import chi2
from sklearn.linear_model import lars_path
from sklearn.model_selection import KFold
from sklearn.covariance import ledoit_wolf
import matplotlib.ticker as mtick
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)
warnings.filterwarnings("ignore", category=RuntimeWarning)
np.seterr(over="ignore", divide="ignore", invalid="ignore", under="ignore")
#%%
# 2. TICKER LIST
# ------------------------------------------------------------------
all_tickers = [
    "A", "AAPL", "ABT", "ADBE", "ADI", "ADM", "ADSK", "AEE", "AEP", "AES",
    "AFL", "AIG", "AJG", "AKAM", "ALB", "ALL", "AMAT", "AMD", "AME", "AMGN",
    "AMT", "AMZN", "ANSS", "AOS", "APA", "APD", "APH", "ARE", "ATO", "AVB",
    "AVY",
    "AXP", "AZO", "BA", "BAC", "BAX", "BBY", "BDX", "BK", "BLK", "BMY", "BRO",
```

```
    "BSX", "BXP", "C", "CAG", "CAH", "CAT", "CB", "CCL", "CHD", "CHRW",
    "CI", "CINF", "CL", "CLX", "CMS", "COF", "COO", "COST", "CPB", "CPRT",
    "CPT", "CSCO", "CSGP", "CSX", "CTAS", "CTSH", "CVS", "D", "DE", "DHI",
    "DHR", "DLTR", "DOV", "DVN", "EBAY", "ECL", "ED", "EOG", "ETN", "ETR",
    "EXPD", "F", "FCX", "FITB", "FMC", "GD", "GE", "GILD", "GIS", "GLW",
    "HAL", "HBAN", "HD", "HIG", "HOLX", "HON", "HRL", "HSY", "HUM", "IBM",
    "INTC", "INTU", "IP", "IPG", "IRM", "IT", "ITW", "JBHT", "JCI", "JNJ",
    "JNPR", "JPM", "K", "KEY", "KIM", "KLAC", "KMB", "KO", "KR", "LLY",
    "LMT", "LNT", "LRCX", "LUV", "MAA", "MAR", "MCD", "MCHP", "MCK", "MDT",
    "MLM", "MMC", "MMM", "MO", "MRK", "MSFT", "MTB", "MTD", "MU",
    "NEM", "NKE", "NOC", "NSC", "NTAP", "NTRS", "NUE", "NVDA", "NVR",
    "NWL", "O", "ODFL", "OKE", "OMC", "ORCL", "ORLY", "OXY", "PAYX",
    "PCAR", "PCG", "PEG", "PEP", "PFE", "PG", "PGR", "PH", "PHM", "PLD",
    "PNC", "PNW", "PPG", "PSA", "PWR", "QCOM", "RCL", "REG", "REGN", "RHI",
    "RJF", "RL", "TECH"
]

# 3. DATA ACQUISITION
↪ ------------------------------------------------------------
print("Connecting to WRDS...")
db = wrds.Connection(wrds_username=WRDS_USER)

tickers_sql = ",".join(f"'{t}'" for t in all_tickers)
query = f"""
    SELECT m.date, s.ticker, m.ret
    FROM   crsp.msf m
    JOIN   crsp.msenames s ON m.permno = s.permno
          AND m.date BETWEEN s.namedt AND s.nameendt
    WHERE  m.date BETWEEN '{START}' AND '{END}'
      AND  s.ticker IN ({tickers_sql})
    ORDER  BY m.date, s.ticker
"""

df = db.raw_sql(query)
db.close()

df['date'] = pd.to_datetime(df['date'])
df = df.drop_duplicates(subset=['date','ticker'], keep='last')
df_pivot = df.pivot(index='date', columns='ticker', values='ret')

# Drop tickers with missing or zero returns
```

```
77  df_clean = df_pivot.dropna(axis=1)
78  df_clean = df_clean.loc[:, (df_clean!=0).any(axis=0)]
79  print(f"Retained {df_clean.shape[1]} tickers over {df_clean.shape[0]}
    ↪  months.")
80
81  R_full  = df_clean.values.astype(float) - RF_MONTH  # EXCESS monthly returns
    ↪  (r_i  r_f)
82  dates   = df_clean.index
83  T, N    = R_full.shape
84
85  # 4. PORTFOLIO HELPERS
    ↪  ------------------------------------------------------------
86  #%%
87  def CovarianceMatrix(R, method="naive"):
88      if method=="naive":
89          return np.cov(R, rowvar=False)
90      if method=="LW":
91          return ledoit_wolf(R)[0]
92      raise ValueError("Unknown covariance method.")
93
94  def compute_theta(R, method="naive"):
95      mu = R.mean(axis=0)
96      S  = CovarianceMatrix(R, method)
97      return mu @ np.linalg.pinv(S) @ mu
98
99  def portfolio_naive(R, sigma):
100     mu      = R.mean(axis=0)
101     S       = CovarianceMatrix(R, "naive")
102     theta   = compute_theta(R, "naive")
103     inv_cov = np.linalg.pinv(S)
104     return (sigma/np.sqrt(theta)) * (inv_cov @ mu)  # unconstrained MV weights
105
106 def _ols(X, y):
107     return np.linalg.pinv(X) @ y
108
109 def _l1_norm(w):
110     return np.sum(np.abs(w))
111
112 def _volatility(w, Sigma_hat):
113     return np.sqrt(w @ Sigma_hat @ w)
114
```

```python
def portfolio_maxser(R, sigma_target, n_folds=10):
    """
    MaxSER: find lasso regression weights
    that match target volatility on average.
    """
    T, _ = R.shape
    Sigma_hat_full = ledoit_wolf(R)[0]
    mu     = R.mean(axis=0)
    theta_hat      = mu @ np.linalg.pinv(Sigma_hat_full) @ mu
    r_c    = sigma_target * (1 + theta_hat) / np.sqrt(theta_hat)
    y_full = np.full(T, r_c)

    # Cross-validate lambda target using K-fold
    kf, lambdas = KFold(n_splits=n_folds), []
    for tr_idx, val_idx in kf.split(R):
        X_tr, X_val = R[tr_idx], R[val_idx]
        y_tr         = np.full(len(tr_idx), r_c)

        # lasso path
        alphas, _, coefs = lars_path(X_tr, y_tr, method="lasso")
        # pick lambda that yields closest vol on hold-out
        gaps = np.abs([
            _volatility(w, ledoit_wolf(X_val)[0]) - sigma_target
            for w in coefs.T
        ])
        lambdas.append(alphas[np.argmin(gaps)])

    lambda_hat = np.mean(lambdas)

    # final fit on full sample
    alpha_full, _, coefs_full = lars_path(R, y_full, method="lasso")
    idx_final        = np.argmin(np.abs(alpha_full - lambda_hat))
    return coefs_full[:, idx_final]

# 5. TRANSACTION-COST BACKTEST
↪ ------------------------------------------------
from typing import Callable, Tuple


def turnover_cost(
    w_prev_star: np.ndarray,
```

```
    w_new:        np.ndarray,
    cost_rate:    float | np.ndarray
) -> float:

    diff = np.abs(w_new - w_prev_star)
    if np.isscalar(cost_rate):
        return float(cost_rate * diff.sum())
    cost_rate = np.asarray(cost_rate, dtype=float)
    if cost_rate.shape != diff.shape:
        raise ValueError("`cost_rate` must be scalar or same length as
        ↪ weights")
    return float(np.dot(cost_rate, diff))


def rolling_backtest_tc(
    R_excess:   np.ndarray,                  # (T,N) matrix of excess returns
    dates:      pd.DatetimeIndex,            # length T
    window_est: int,                         # look-back window for estimation
    hold:       int,                         # re-balance/hold horizon
    sigma_tgt:  float,                       # target volatility (monthly)
    strat_fn:   Callable[[np.ndarray, float], np.ndarray],
    cost_rate:  float | np.ndarray = 0.001, # proportional fee (10bp default)
    rf_month:   float = 0.0                  # constant r_f in *raw* monthly
    ↪ terms
) -> Tuple[np.ndarray, pd.DatetimeIndex]:

    T, N = R_excess.shape
    rets, idx, wts = [], [], []

    # ---------- initialise with first optimal weights (start fully invested)
    ↪ ---
    w_prev = strat_fn(R_excess[:window_est], sigma_tgt)

    # rolling loop
    ↪ ----------------------------------------------------------------
    t = window_est
    while t + hold <= T - 1:
        # ------- 1. allow drift over the *current* month
        ↪ ----------------------
        r_raw_t  = R_excess[t] + rf_month           # convert to raw returns
        cash_prev = 1.0 - w_prev.sum()              # implicit RF position (±)
```

```
        numer  = w_prev * (1.0 + r_raw_t)          # risky dollar sleeve
        nav_growth = cash_prev * (1.0 + rf_month) + numer.sum()
        w_prev_star = numer / nav_growth           # w(t+)  in the paper

        # ------- 2. compute new target weights based on look-back window
        ↪ -------
        w_new = strat_fn(R_excess[t - window_est : t], sigma_tgt)

        # ------- 3. proportional turnover cost  _t  (Eq.6)
        ↪ --------------------
        tau_t = turnover_cost(w_prev_star, w_new, cost_rate)

        # ------- 4. hold `w_new` for `hold` months, net of cost in month-1
        ↪ ------
        for h in range(1, hold + 1):
            r_gross = np.dot(R_excess[t + h], w_new)        # excess port.
            ↪ ret.
            if h == 1:                                      # pay fee once
                r_net = (1.0 - tau_t) * (1.0 + r_gross) - 1.0
            else:
                r_net = r_gross
            rets.append(r_net)
            idx.append(dates[t + h])

        # ------- 5. carry new weights forward
        ↪ ---------------------------------
        w_prev = w_new.copy()
        wts.append(w_new)                                 # store w every month
        t += hold

    return np.array(rets, dtype=float), pd.DatetimeIndex(idx),wts
#%%
# 6. RUN BACKTESTS
↪ ------------------------------------------------------------
print("Running backtests with TC and financing...")
ret_naive, idx_n,w_n = rolling_backtest_tc(R_full, dates,
                                           WIN_EST, HOLD, SIGMA_TGT,
                                           portfolio_naive, TC_RATE)

ret_maxser, idx_m,w_m = rolling_backtest_tc(R_full, dates,
                                             WIN_EST, HOLD, SIGMA_TGT,
```

```
                                        portfolio_maxser, TC_RATE)

#%%
df_ret = pd.DataFrame({
    "Naive": pd.Series(ret_naive, index=idx_n),
    "MaxSER": pd.Series(ret_maxser, index=idx_m)
})

# 7. OOS METRICS
# ----------------------------------------------------------------
vol_oos = df_ret.rolling(WIN_EVAL).std(ddof=1)
shp_oos = df_ret.rolling(WIN_EVAL).apply(lambda x: x.mean()/x.std(ddof=1),
    raw=True)

# 95% CI for volatility (²)
alpha = 0.05
df_    = WIN_EVAL - 1
var_oos = df_ret.rolling(WIN_EVAL).var(ddof=1)
chi2_low  = chi2.ppf(1-alpha/2, df_)
chi2_high = chi2.ppf(alpha/2,  df_)
var_low   = (df_*var_oos)/chi2_low
var_high  = (df_*var_oos)/chi2_high
vol_low, vol_high = np.sqrt(var_low), np.sqrt(var_high)

# 8. PLOTS
# ----------------------------------------------------------------------
# Cumulative returns
plt.figure(figsize=(12, 6))

cum = df_ret.add(1).cumprod() - 1                # decimal form
cum_floor = cum.mask((cum <= -1).cummax(), -1)
cum_pct = cum_floor * 100                        # convert to %

# emphasize MaxSER
ax = cum_pct.plot(style=['--', '-'])
ax.lines[0].set_linewidth(1)
ax.lines[1].set_linewidth(2)

plt.title("OOS Cumulative Returns (Net of TC)")
plt.ylabel("Cumulative Return (%)")

```

```
# centre y-axis on MaxSER
maxser = cum_pct["MaxSER"]
mid  = (maxser.max() + maxser.min()) / 2
span = (maxser.max() - maxser.min()) * 1.5
ax.set_ylim(mid - span/2, mid + span/2)

plt.gca().yaxis.set_major_formatter(mtick.PercentFormatter())
plt.grid(ls='--', alpha=0.4)
plt.legend(title="Strategy")
plt.tight_layout()
plt.show()


plt.figure(figsize=(10,5))
vol_oos.plot(ax=plt.gca())
plt.axhline(SIGMA_TGT, ls='--', color='k', label='Target ')
plt.title("Rolling Volatility (Net of TC)")
plt.legend(); plt.grid(ls='--', alpha=0.4); plt.tight_layout(); plt.show()

plt.figure(figsize=(10,5))
shp_oos.plot(ax=plt.gca()); plt.axhline(0, ls='--', color='k', lw=0.5)
plt.title("Rolling Sharpe (Net of TC)"); plt.grid(ls='--', alpha=0.4)
plt.tight_layout(); plt.show()

print("\n--- Mean OOS Metrics (Net) ---")
print("Volatility:\n", vol_oos.mean())
print("Sharpe:\n", shp_oos.mean())
np.mean(ret_maxser)/np.std(ret_maxser)
np.mean(ret_maxser)/np.std(ret_maxser)
mu_m = np.mean(ret_maxser)
mu_n = np.mean(ret_naive)
print(f"MaxSER expected return: {mu_m:.4f}")
print(f"Naive expected return: {mu_n:.4f}")
std_maxser = np.std(ret_maxser)
std_naive  = np.std(ret_naive)
print(f"MaxSER Std: {std_maxser:.4f}")
print(f"Naive Std: {std_naive:.4f}")
print(f"MaxSER Sharpe: {np.mean(ret_maxser)/std_maxser:.4f}")
print(f"Naive Sharpe: {np.mean(ret_naive)/std_naive:.4f}")
#%%
# Note: Total TC is embedded in returns; for reference we can still sum _t:
```

```python
tc_naive = np.sum([
    turnover_cost(
        np.zeros_like(R_full[0]),
        portfolio_naive(R_full[k-WIN_EST:k], SIGMA_TGT),cost_rate=TC_RATE
    )
    for k in range(WIN_EST, len(R_full), HOLD)
])

tc_maxser = np.sum([
    turnover_cost(
        np.zeros_like(R_full[0]),
        portfolio_maxser(R_full[k-WIN_EST:k], SIGMA_TGT),cost_rate=TC_RATE
    )
    for k in range(WIN_EST, len(R_full), HOLD)
])

print(f"\nTotal proportional TC paid (Naive): {tc_naive:.6f}")
print(f"Total proportional TC paid (MaxSER): {tc_maxser:.6f}")

# %%
```